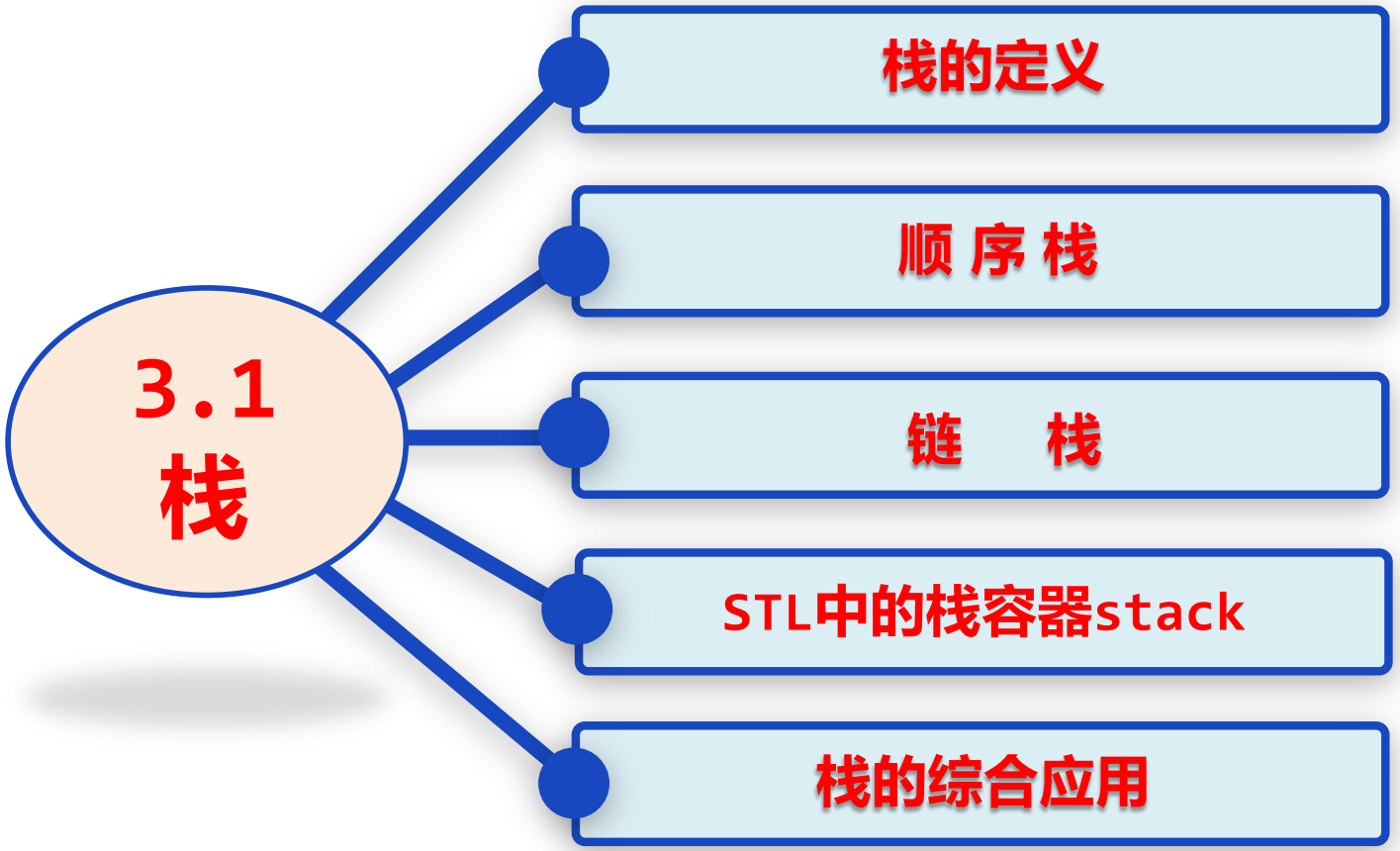
第3章 栈和队列

提纲

CONTENTS

3.1 栈

3.2 队列


3.1
栈
栈的定义
顺 序 栈
链　　栈
STL中的栈容器stack
栈的综合应用

场景1：文字编辑器的"撤销功能"

- 当同学们在Word中误删了一段文字，第一个反应是什么？没错按下Ctrl+Z！这个神奇的功能是如何实现的呢？
- 其背后正是栈结构。每次编辑操作都会被压入栈中，撤销时弹出最近的操作，就像一层层剥开洋葱皮。

```
//算法示意用户输入“hello”字符串及回退的过程
栈类型 history; // 操作历史栈
// 用户输入"Hello"
history.入栈("");        // 初始状态
history.入栈("H");
history.入栈("He");
history.入栈("Hel");
history.入栈("Hell");
history.入栈("Hello");

// 执行一次撤销（Ctrl+Z）
history.出栈(); // 当前内容回退到"Hell"
```

- 拓展：如何同时实现“撤销”和“重做”？

场景2：浏览器前进后退

- 当我们浏览网页时，后退按钮总能带我们回到上一个访问的页面。
- 这也是栈的经典应用！

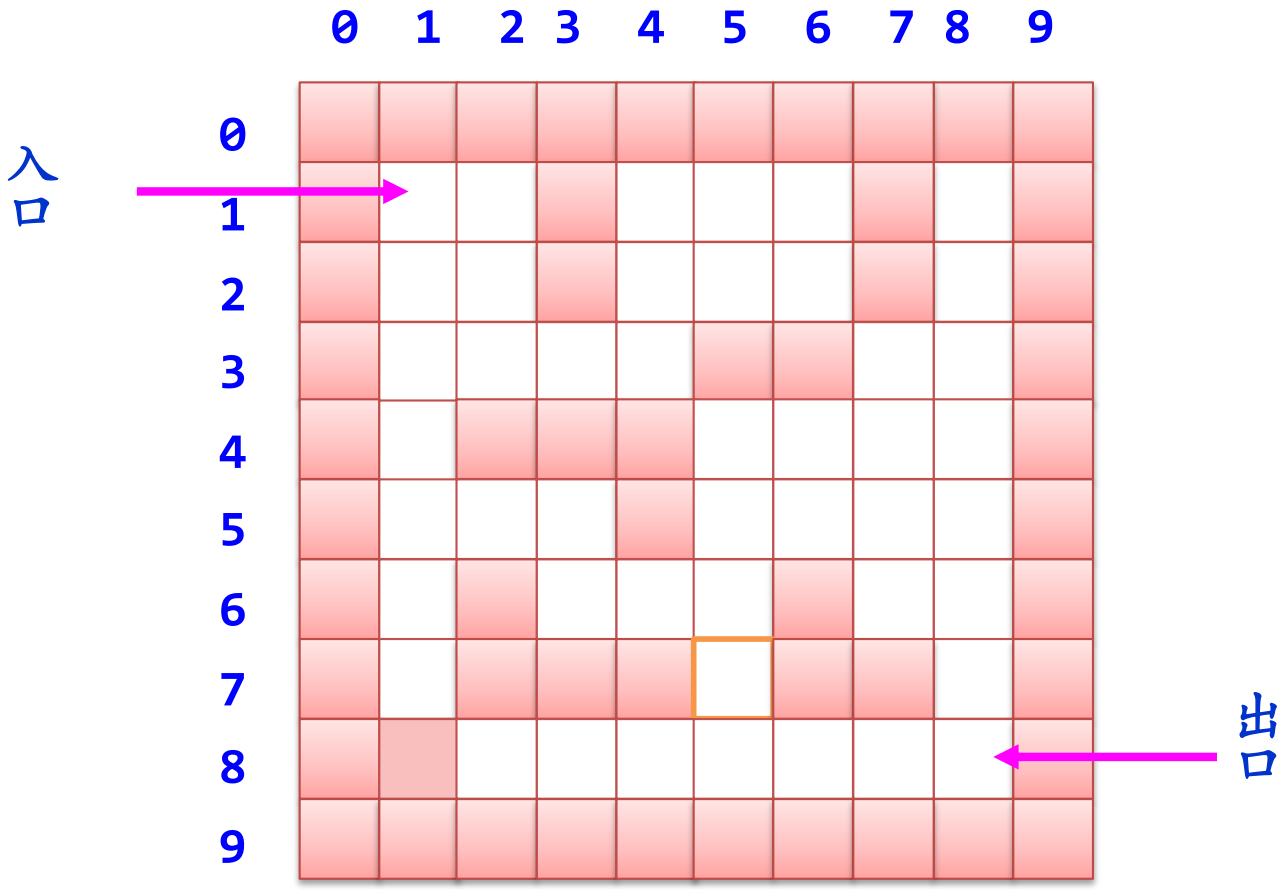
场景3：编译器括号匹配检测

```
//下面这段代码有什么错误？
int main() {
    if ((x > 0) && (y < 10)){
        cout << "Error!";          // 缺少一个右括号
    }
}
```

- 如何让计算机自动检测这类错误？

```
栈类型 brackets;
for (char c : code) {
    if (c == '(' || c == '{' || c == '[') {
        brackets.入栈(c);
    } else if (c == ')') {
        if (brackets.非空() || brackets.取栈顶() != '(')
            throw error;
        brackets.出栈();
    }
    // 其他括号类似处理
}
```

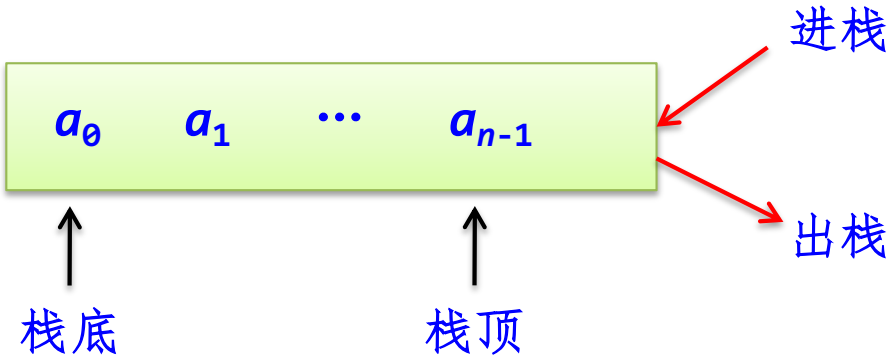
场景4：走迷宫

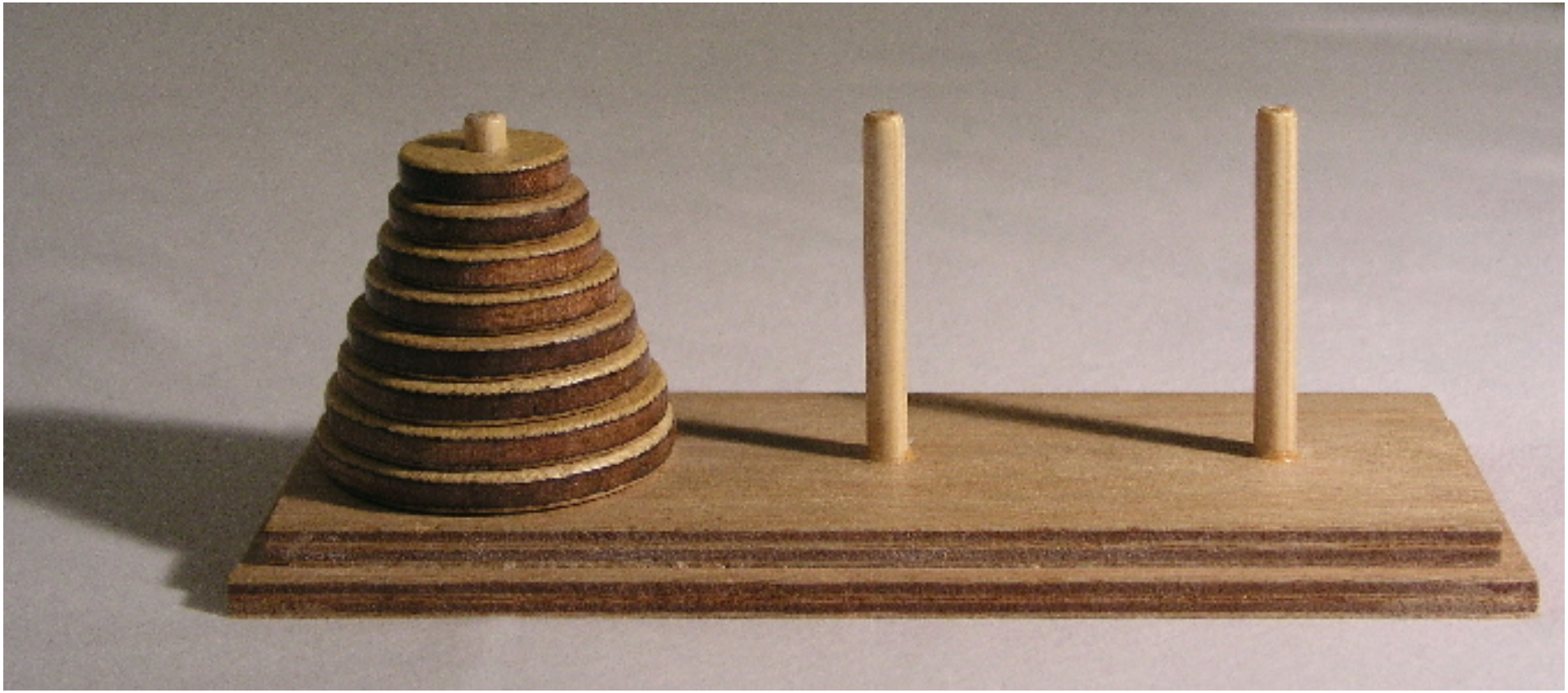


- 走错路的时候如何回溯？

3.1.1 栈的定义

- 栈（stack）是一种只能在同一端进行插入或删除操作的线性表。
- 表中允许进行插入、删除操作的一端称为栈顶（top），表的另一端称为栈底（bottom）。
- 栈的插入操作通常称为进栈或入栈（push），栈的删除操作通常称为退栈或出栈（pop）。





后放置的盘片先取出来

栈的主要特点：

- 后进先出，即后进栈的元素先出栈。
- 每次进栈的元素都作为新栈顶元素，每次出栈的元素只能是当前栈顶元素。
- 栈也称为后进先出表或者先进后出表。

ADT Stack

{

数据对象：

D={a_i | 0≤i≤n-1，n≥0，元素a_i为T类型}

数据关系：

R={r}

r={<a_i，a_{i+1}> | a_i，a_{i+1}∈D， i=0，…，n-2}

基本运算：

empty()：判断栈是否为空，若空栈返回真；否则返回假。

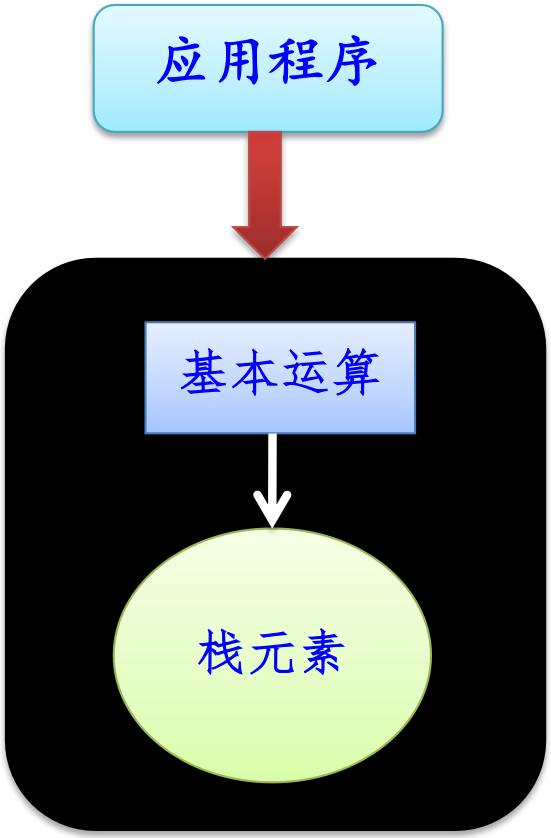
push(T e)：进栈操作，将元素e插入到栈中作为栈顶元素。

pop(T& e)：出栈操作。

gettop(T& e)：取栈顶操作。

}

栈抽象数据类型 = 线性结构 + 栈的基本运算


应用程序
基本运算
栈元素

示例

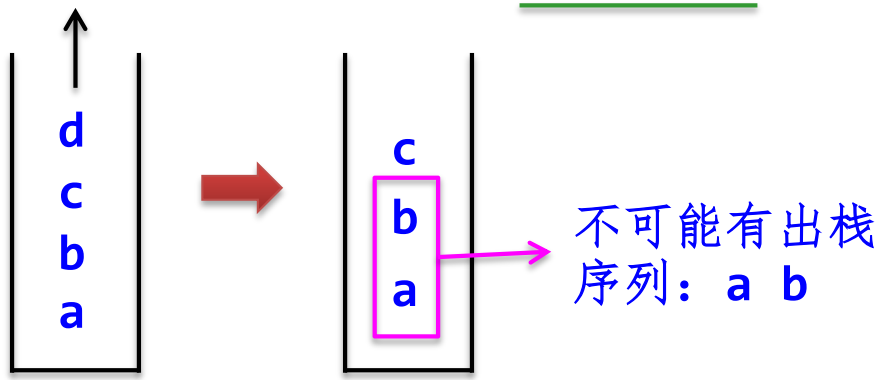
一个栈的进栈序列是a、b、c、d、e，则栈的不可能的输出序列是（　）。

A.edcba　　B.decba

C.dceab　　D.abcde

方法1：用栈模拟进行判断

考虑 C.dceab



示例

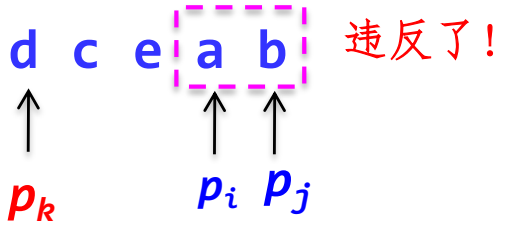
一个栈的进栈序列是a、b、c、d、e，则栈的不可能的输出序列是（　）。

A.edcba　　B.decba

C.dceab　　D.abcde

方法2：利用判断准则

判断准则：输入序列为1，2，…，n，利用一个栈得到输出序列（p_1，p_2，…，p_n），（p_1，p_2，…，p_n）是1，2，…，n的一种排列，合法输出的充分必要条件是不存在这样的i、j、k满足$i<j<k$的同时也满足 $p_k<p_i<p_j$



输出序列中在k后面的比k小的数一定按严格递减排列.否则出栈系列不合法

示例

已知一个栈的进栈序列是1，2，3，…，n，其输出序列是p_1，p_2，…，p_n，若$p_1=n$，则p_i的值为（　）。

A.i　　B.$n-i$　　C.$n-i+1$　　D.不确定

1，2，3，…，n ➡ n，…

输出序列唯一

$p_1=n$

$p_2=n-1$

$p_3=n-2$

…

$p_n=1$

$p_i+i=n+1$即$p_i=n-i+1$

2013年全国硕士研究生入学统一考试题

示例

2.一个栈的入栈序列为1,2,3，…,n ，其出栈序列是$p_1,p_2,p_3,\cdots,p_n$。若$p_2=3$，则p_3可能取值的个数是（　）。

A.$n-3$　　B.$n-2$　　C.$n-1$　　D.无法确定

n，…,4



1进，1出，2进，3进，3出，2出，…



p_1 p_2 p_3

或者　1进，2进，2出，3进，3出，1出，…



p_1 p_2 p_3

或者　1进，2进，2出，3进，3出，4进，4出，…，1出



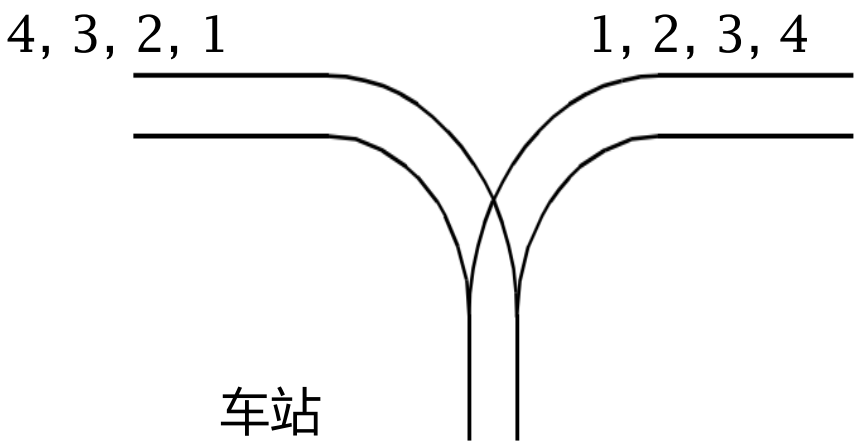
p_1 p_2 p_3

…　p_3除了3外都可能！

示例

火车进出栈问题

编号为1，2，…，*n* 的*n*辆火车依次进站，给定一个*n*的排列，判断是否为合法的出站顺序？



若入栈顺序为1,2,3,4的话，则出栈的顺序可以有哪些？

1234

1243

1324

1342

1423

1432

2134

2143

……



火车进出栈问题

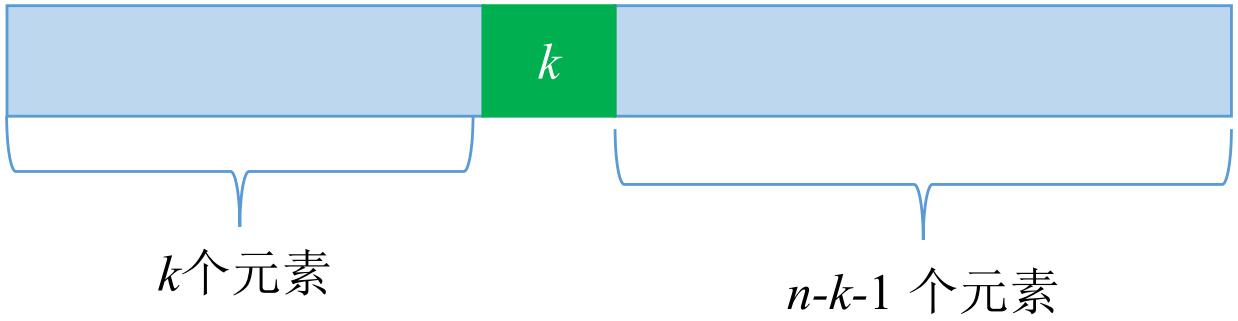
编号为1，2，…，*n* 的*n*辆火车依次进站，给定一个*n*的排列，判断是否为合法的出站顺序？

```
#include <stack>
#include <vector>
using namespace std;

bool isValidExitOrder(int n, vector<int>& target){
    stack<int> station;
    int j = 0; // 指向target的当前检查位置
    for (int i = 1; i <= n; ++i) {
        station.push(i); // 火车进站
        // 尽可能弹出栈顶匹配的车辆
        while (!station.empty()&& j<n && station.top() == target[j]){
              station.pop();
              ++j;
        }
     }
     return j == n; // 是否所有车辆都已正确出站
}
```

Catalan数

给定一个入栈序列，序列长度为**n**，请问有多少种出栈序列？



$$f(N)=\sum_{k=0}^{n-1} f(k)\times f(N-1-k)$$

计算方法

(1) $C_n = C_0C_{n-1} + C_1C_{n-2} + \ldots + C_{n-2}C_1 + C_{n-1}C_0 = \sum C_kC_{n-k}, \quad C_0 = 1$

(2) $C_n = \dfrac{4n-2}{n+1}C_{n-1}, \quad C_0 = 1$

(3) $C_n = \dfrac{1}{n+1}\binom{2n}{n} = \dfrac{(2n)!}{(n+1)!n!}$

部分Catalan数：1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, 2674440, 9694845, 35357670 ......

Catalan
示例

棋盘问题

一个n行n列的棋盘，从左下角走到右上角，一直在对角线右下方走，不穿过主对角线，走法有多少种？

例如n=4时，有14种走法。

Catalan
示例

括号问题

用n个左括号和n个右括号，组成一串字符串，有多少种合法的组合？

例如，“（ ）（ ）（ （ ） ）”是合法的，而“（ ） ）（ （ ）”是非法的。

显然，合法的括号组合是：任意前k个括号组合，左括号的数量大于等于右括号的数量。

解：

定义左括号为0，右括号为1。问题转化为：n个0和n个1组成的序列，任意前k个序列中，0的数量都大于等于1的数量。

模型和棋盘问题一样。

Catalan
示例

出栈序列问题

给定一个以字符串形式表示的入栈序列，请求出一共有多少种可能的出栈顺序？

例如入栈序列为：{1 2 3}，则出栈序列一共有五种：{1 2 3}、{1 3 2}、{2 1 3}、{2 3 1}、{3 2 1}

解：

合法的序列：对于出栈序列中的每一个数字，在它后面的、比它小的所有数字，一定是按递减顺序排列的。例如，{3 2 1}是合法的，3出栈之后，比它小的后面的数字是：{2 1}，且这个顺序是递减顺序。而{3 1 2} 是不合法的，因为在3后面的数字{1 2}，是一个递增的顺序。

一个栈的进栈序列是a、b、c、d、e，则栈的不可能的输出序列是（ ）。

A.edcba　　B.decba

C.dceab　　D.abcde

Catalan
示例

出栈序列问题

给定一个以字符串形式表示的入栈序列，请求出一共有多少种可能的出栈顺序？

例如入栈序列为：{1 2 3}，则出栈序列一共有五种：{1 2 3}、{1 3 2}、{2 1 3}、{2 3 1}、{3 2 1}

解：

对于每一个数来说，必须进栈一次、出栈一次。定义进栈操作为0，出栈操作为1。n个数的所有状态对应n个0和n个1组成的序列。出栈序列，即要求进栈的操作数大于等于出栈的操作数。

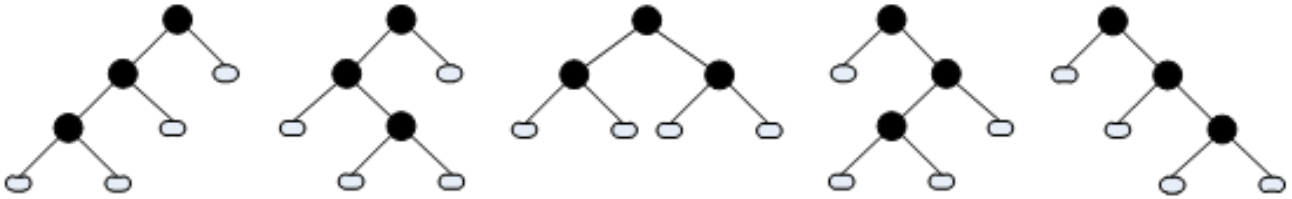
问题转化为：由n个1和n个0组成的2n位二进制数，任意前k个序列，0的数量大于或等于1的数量。结果是catalan数。

Catalan
示例

二叉树问题

n个结点构成的二叉树，共有多少种情况？

例如，有3个结点（图中的黑点）的二叉树，可以构成5种二叉树：



解：

$$C_n = C_0C_{n-1} + C_1C_{n-2} + \ldots + C_{n-2}C_1 + C_{n-1}C_0 = \sum C_kC_{n-k}, \quad C_0 = 1$$

C_0C_{n-1}：右子树有0个结点 + 左子树有n-1个结点；

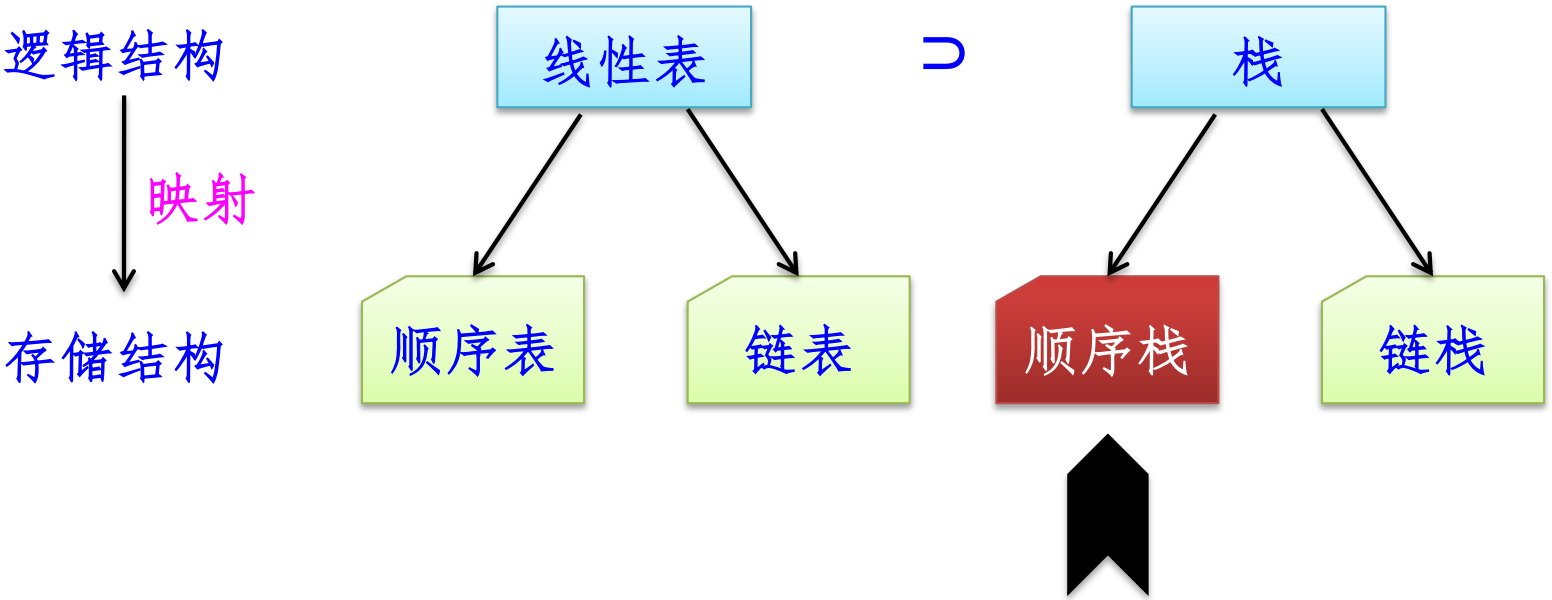
C_1C_{n-2}：右子树有1个结点 + 左子树有n-2个结点；

……

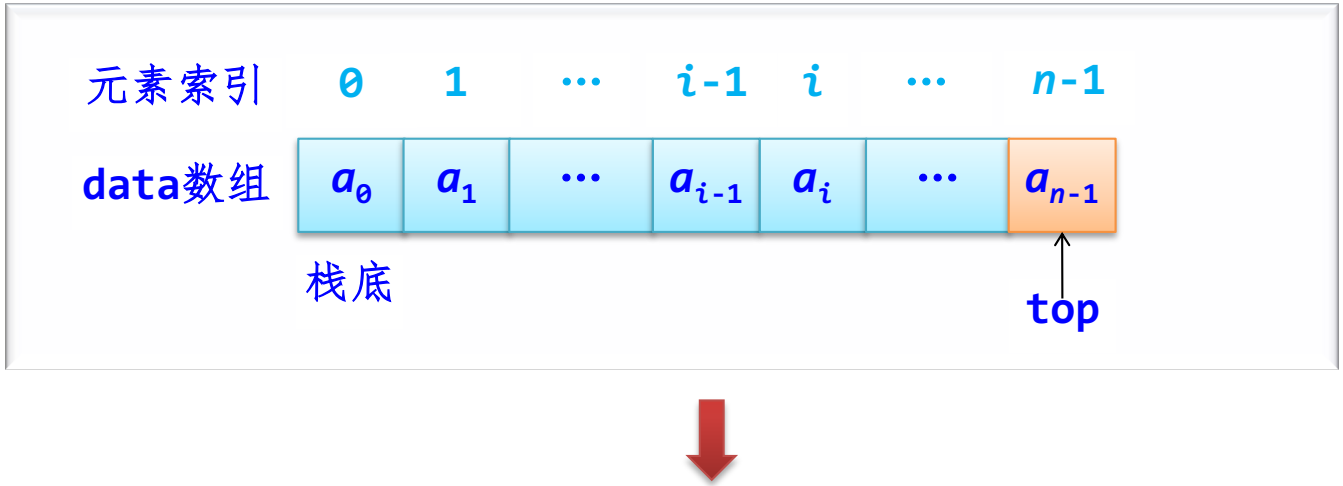
$C_{n-1}C_0$：右子树有n-1个结点 + 左子树有0个结点；

3.1.2 栈的顺序存储结构及其基本运算算法实现

栈的实现方式

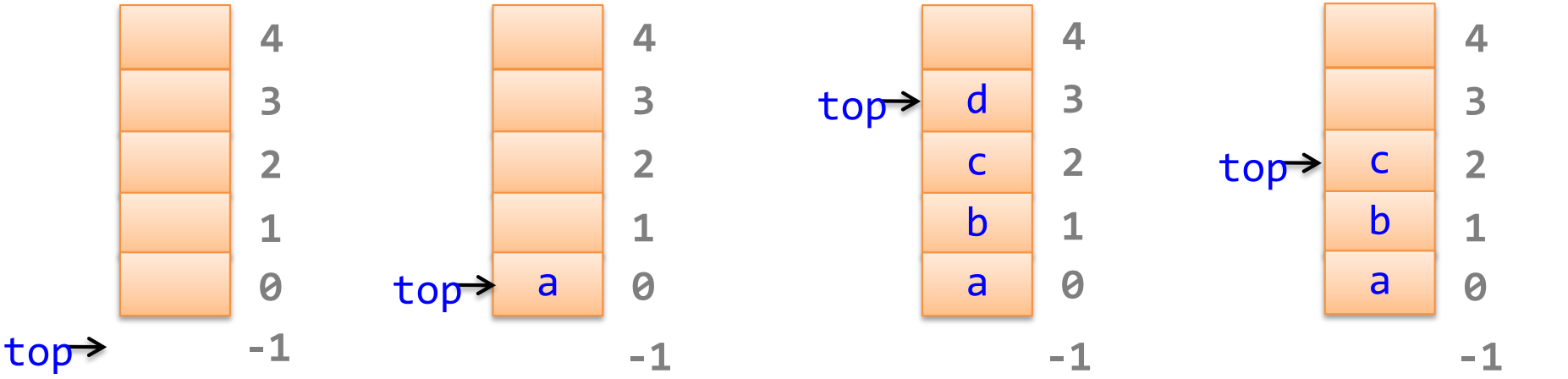


顺序栈实现



- 由于栈顶是动态变化的，为此设置一个栈顶指针top以反映栈状态，约定top总是指向栈顶元素。
- 为了简单，这里的data数组采用固定容量（容量为MaxSize）分配方式，并且置data[0]端作为栈底，另外一端data[MaxSize-1]作为栈顶，其中的元素个数恰好为top+1。

顺序栈（MaxSize=5）



（a）空栈　（b）元素a进栈　（c）元素b、c、d进栈　（d）元素d出栈

顺序栈的四要素如下（初始时top=-1）：

① 栈空条件：top==-1。

② 栈满条件：top==MaxSize-1。

③ 元素e进栈操作：top++;data[top]=e。

④ 出栈操作：e=data[top];top--。

顺序栈类模板SqStack

```
template <typename T>
class SqStack                          //顺序栈类模板
{  T* data;                            //存放栈中元素
   int top;                            //栈顶指针
   //栈的基本运算算法
};
```

（1）顺序栈的初始化和销毁

（2）判断栈是否为空empty()

（3）进栈push(e)

（4）出栈pop()

（5）取栈顶元素gettop()

顺序栈的基本运算算法

（1）顺序栈的初始化和销毁

```
SqStack()                          //构造函数
{   data=new T[MaxSize];           //为data分配容量为MaxSize的空间
    top=-1;                        //栈顶指针初始化
}
```

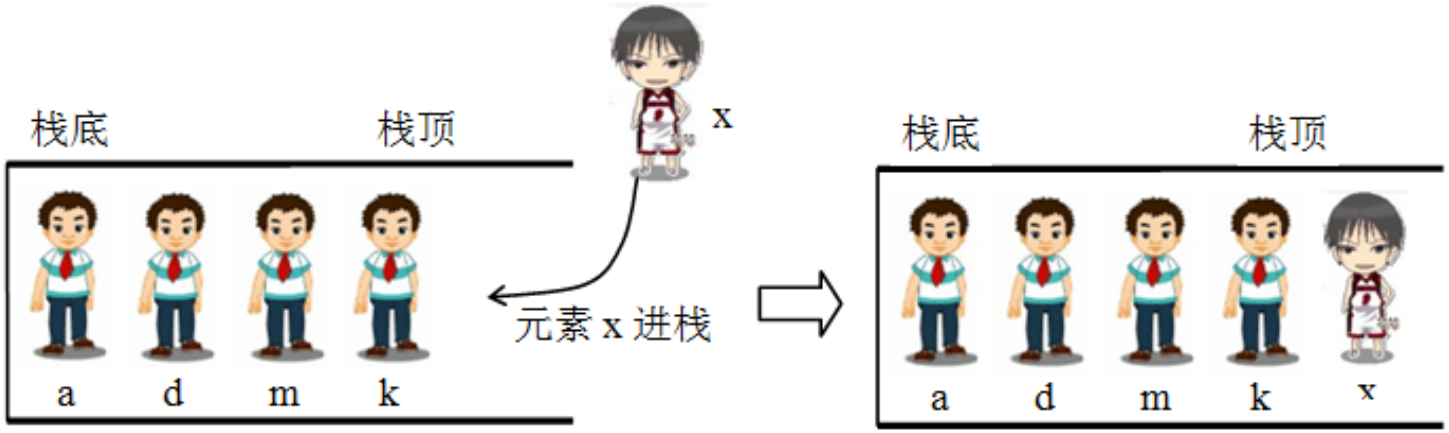
```
~SqStack()                         //析构函数
{
    delete [] data;
}
```

（2）判断栈是否为空empty()

```
bool empty()                //判断栈是否为空
{
    return top==-1;
}
```

（3）进栈push(e)

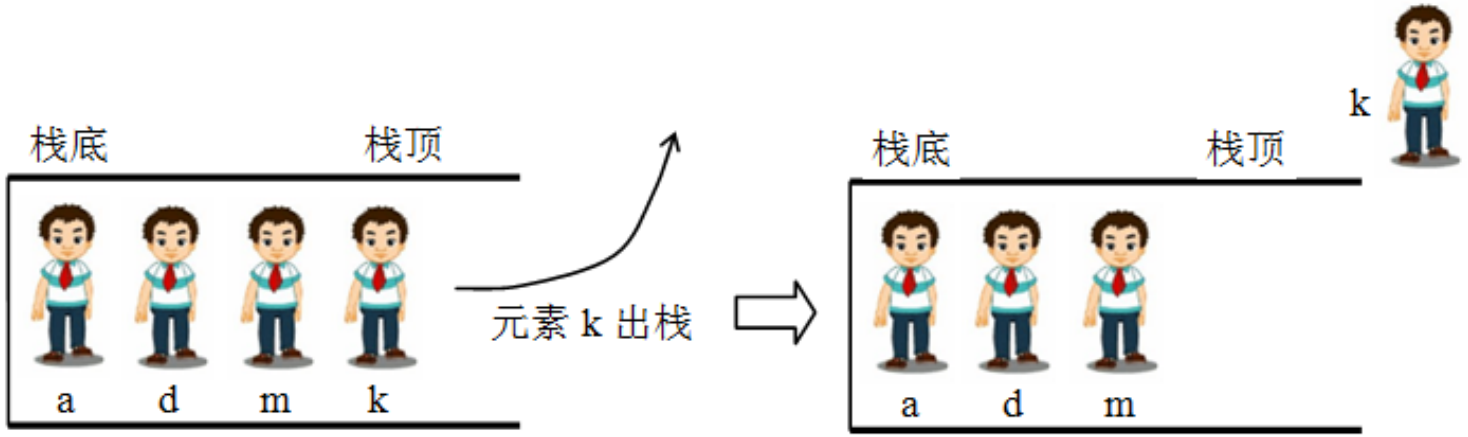
元素进栈只能从栈顶进，不能从栈底或中间位置进栈



```
bool push(T e)                    //进栈算法
{  if (top==MaxSize-1)            //栈满时返回false
      return false;
   top++;                         //栈顶指针增1
   data[top]=e;                   //将e进栈
   return true;
}
```

（4）出栈pop()

元素出栈只能从栈顶出，不能从栈底或中间位置出栈



```
bool pop(T& e)                          //出栈算法
{   if (empty())                        //栈为空的情况，即栈下溢出
        return false;
    e=data[top];                        //取栈顶指针元素的元素
    top--;                              //栈顶指针减1
    return true;
}
```

（5）取栈顶元素gettop()

```
bool gettop(T& e)                    //取栈顶元素算法
{   if (empty())                     //栈为空的情况，即栈下溢出
        return false;
    e=data[top];                     //取栈顶指针位置的元素
    return true;
}
```



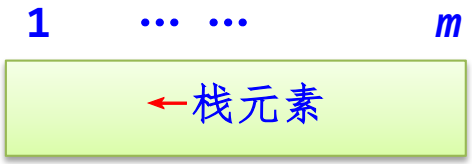
顺序栈的几个问题

问题1：若采用数组data[1..*m*]存放栈元素，回答以下问题：

（1）只能以data[1]端作为栈底吗？

（2）为什么不能以data数组的中间位置作为栈底？

答：（1）也可以将data[*m*]端作为栈底。



（2）栈中元素是从栈底向栈顶方向生长的，如果以data数组的中间位置作为栈底，那么栈顶方向的另外一端空间就不能使用，造成空间浪费，所以不能以data数组的中间位置作为栈底。

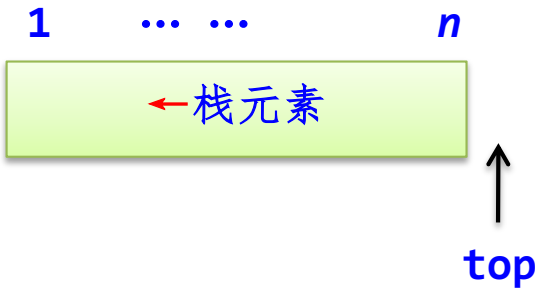
问题2（常规顺序栈）：

若一个栈用数组data[1..*n*]存储，初始栈顶指针top为*n*+1，则以下元素*x*进栈的正确操作是（　）。

A.top++; data[top]=*x*;　B.data[top]=*x*; top++;

C.top--; data[top]=*x*;　D.data[top]=*x*; top--;

答：初始栈顶指针top为*n*+1，说明data[*n*]端作为栈底，在进栈时top应递减，由于不存在data[*n*+1]的元素，所以在进栈时应先将top递减，再将*x*放在top处（top指向栈顶元素）。答案为C。

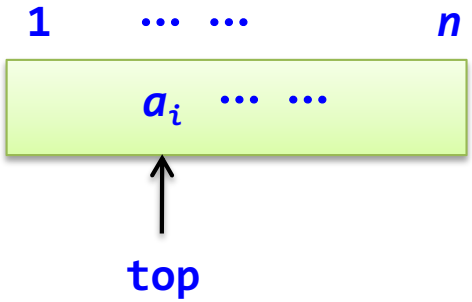


若一个栈用数组data[1..*n*]存储，初始栈顶指针top为*n*+1，则以下元素*x*出栈的正确操作是（　）。

A.*x*=data[top]; top++;　B.top++; *x*=data[top];

C.*x*=data[top]; top--;　D.top--; *x*=data[top];

答：进栈操作是：top--;data[top]=*x*（ top指向栈顶元素）；出栈操作与进栈操作相反，应该为*x*=data[top]; top++;。题答案为A。

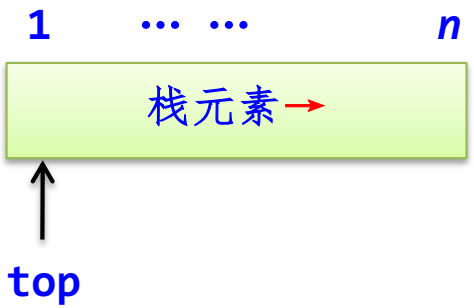


若一个栈用数组data[1..*n*]存储，初始栈顶指针top为1，则以下元素*x*进栈的正确操作是（　）。

A.top++; data[top]=*x*　　B.data[top]=*x*; top++;

C.top--; data[top]=*x*;　D.data[top]=*x*; top--;

答：初始栈顶指针top为1，说明data[1]端作为栈底，在进栈时top应递增，由于存在data[1]的元素，所以在进栈时应先将*x*放在top处，再top递增（top指向栈顶元素的前一个位置！）。答案为B。

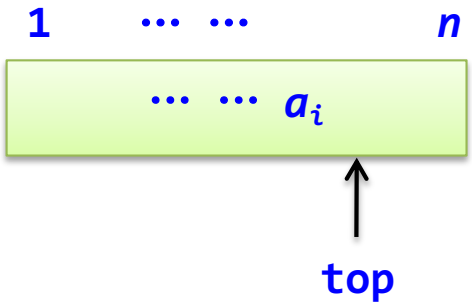


若一个栈用数组data[1..*n*]存储，初始栈顶指针top为1，则以下元素*x*出栈的正确操作是（　）。

A.*x*=data[top]; top++;　B.top++; *x*=data[top];

C.*x*=data[top]; top--　D.top--; *x*=data[top];

答：进栈操作是：B.data[top]=*x*; top++;（top指向栈顶元素的前一个位置）。出栈操作与进栈操作相反，应该为top--; *x*=data[top]。题答案为D。



3.1.3 顺序栈的应用算法设计示例

【例3.4】设计一个算法利用顺序栈检查用户输入的表达式中括号是否配对（假设表达式中可能含有圆括号、中括号和大括号）。并用相关数据进行测试。

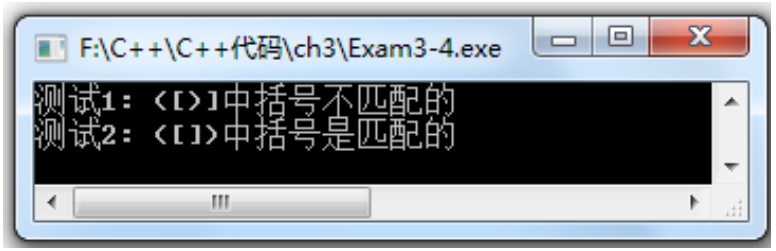



```
#include "SqStack.cpp"                          //包含顺序栈类模板的定义
bool isMatch(string str)                        //判断表达式各种括号是否匹配的算法
{  SqStack<char> st;                            //建立一个顺序栈
   int i=0;
   char e;
   while (i<str.length())
   {  if (str[i]=='(' || str[i]=='[' || str[i]=='{')
         st.push(str[i]);     //遇到将左括号,均进栈
```

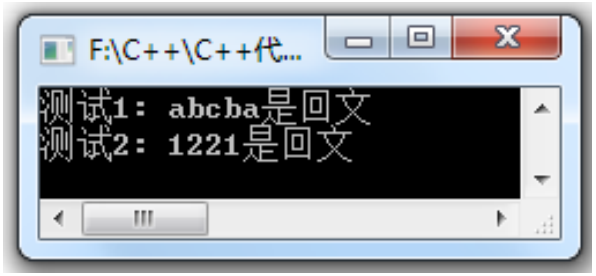
```
        else
        {   if (str[i]==')')                    //遇到')'
            {   if (st.empty())                 //栈空时返回false
                    return false;
                st.pop(e);                      //出栈元素e
                if (e!='(')                     //栈顶不是匹配的'(',返回false
                    return false;
            }
            if (str[i]==']')                    //遇到']'
            {   if (st.empty())                 //栈空时返回false
                    return false;
                st.pop(e);                      //出栈元素e
                if (e!='[')                     //栈顶不是匹配的'[',返回false
                    return false;
            }
            if (str[i]=='}')                    //遇到'}'
            {   if (st.empty())                 //栈空时返回false
                    return false;
                st.pop(e);                      //出栈元素e
                if (e!='{')                     //栈顶不是匹配的'{',返回false
                    return false;
            }
        }
        i++;                                    //继续遍历str
    }
    return st.empty();
}
```

```
int main()
{  cout << "测试1: ";
   string str="([)]";
   if (isMatch(str))
      cout << str << "中括号是匹配的" << endl;
   else
      cout << str << "中括号不匹配的" << endl;

   cout << "测试2:";
   str="([])";
   if (isMatch(str))
      cout << str << "中括号是匹配的" << endl;
   else
      cout << str << "中括号不匹配的" << endl;
   return 0;
}
```



【例3.5】设计一个算法利用顺序栈判断用户输入的字符串表达式是否为回文。并用相关数据进行测试。





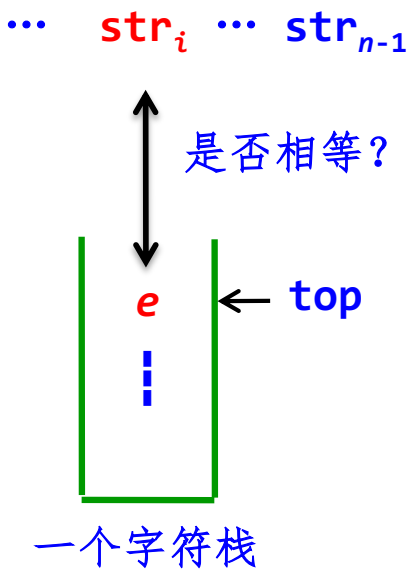
用str存放表达式，其中含n个字符。若str的前半部分的反向序列与str的后半部分相同，则是回文，否则不是回文。判断过程如下：

① 用i从头开始遍历str，将前半部分字符依次进栈。

② 若n为奇数，i增1跳过中间的字符。

③ i继续遍历其他后半部分字符，每访问一个字符，则出栈一个字符，两者进行比较，如图所示，若不相等返回False。

④ 当str遍历完毕返回True。



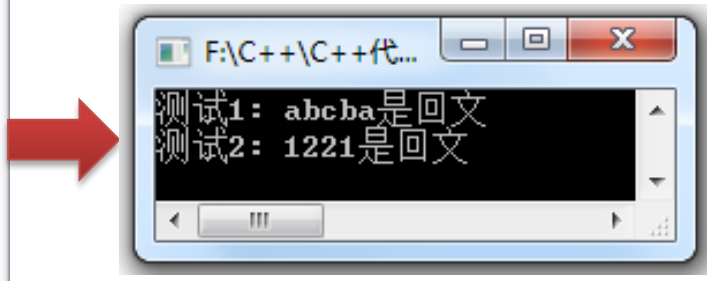
```
#include "SqStack.cpp"                  //包含顺序栈类模板的定义
bool isPalindrome(string str)           //判断是否为回文的算法
{  SqStack<char> st;                    //建立一个顺序栈
   char e;
   int i=0;
   while (i<str.length()/2)             //将str前半部分字符进栈
   {  st.push(str[i]);
      i++;                              //继续遍历str
   }

   if (str.length()%2==1)               //str长度为奇数时
      i++;                              //跳过中间的字符

   while (i<str.length())               //遍历str的后半部分字符
   {  if (st.empty()) false;            //栈空时返回false
      st.pop(e);                        //出栈元素e
      if (e!=str[i]) return false;      //str[i]≠出栈字符返回false
      i++;
   }
   return true;                         //是回文返回true
}
```

```
int main()
{   cout << "测试1: ";
    string str="abcba";
    if (isPalindrome(str))
        cout << str << "是回文" << endl;
    else
        cout << str << "不是回文" << endl;

    cout << "测试2: ";
    str="1221";
    if (isPalindrome(str))
        cout << str << "是回文" << endl;
    else
        cout << str << "不是回文" << endl;
    return 0;
}
```



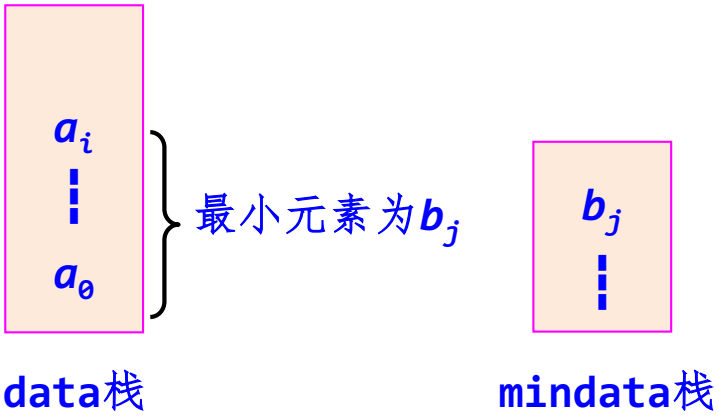
【例3.6*】设计最小栈。定义栈的数据结构，添加一个Getmin()函数用于返回栈中的最小元素。要求函数Getmin()、push以及pop的时间复杂度都是O(1)。例如：

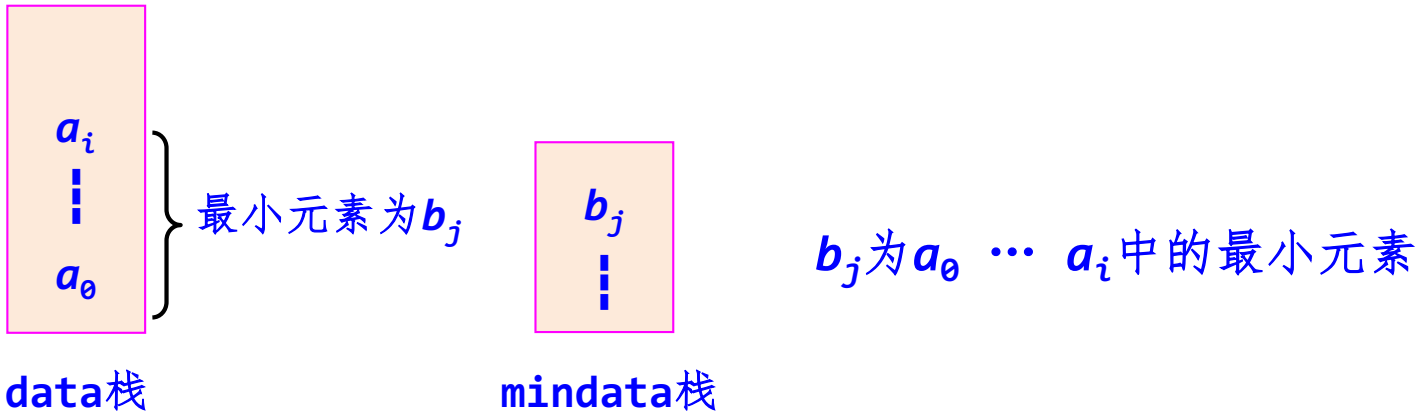
```
push(5); #栈元素：（5）              最小元素：5
push(6); #栈元素：（6，5）           最小元素：5
push(3); #栈元素：（3，6，5）        最小元素：3
push(7); #栈元素：（7，3，6，5）     最小元素：3
pop();   #栈元素：（3，6，5）        最小元素：3
pop();   #栈元素：（6，5）           最小元素：5
```



设计满足题目要求的顺序栈类为STACK：

- 含data和mindata两个数组，data数组表示data栈（主栈），mindata数组表示mindata栈，后者作为存放当前最小元素的辅助栈。
- 当元素a_0，a_1，…，a_i（$i \geq 1$）进栈到data栈后，min栈的栈顶元素b_j为a_0，a_1，…，a_i中的最小元素（含后进栈的重复最小元素），如下图所示。





STACK类的主要运算算法设计如下：

- Getmin()函数用于返回栈中的最小元素，其操作是取mindata栈的栈顶元素。
- 进栈函数push(x)的操作是，当data栈空或者进栈元素x小于等于当前栈中最小元素（即x≤Getmin()）时，则将x进mindata栈。最后将x进data栈。
- 出栈函数pop()的操作是，当data栈不空时，从data栈出栈元素x，若mindata栈的栈顶元素等于x，则同时从mindata栈出栈x。最后返回x。
- 取栈顶函数gettop()的操作是，当data栈不空时，返回data栈的栈顶元素。

```
#include<iostream>
using namespace std;
const int MaxSize=100;                    //栈中最多元素个数
template <typename T>
class STACK                               //含Getmin()的栈类
{   T data[MaxSize];                      //存放主栈中元素，初始为空
    T mindata[MaxSize];                   //存放min栈中元素，初始为空
    int top;
    int mintop;
public:
    STACK():top(-1),mintop(-1) {}         //构造函数
```

```
private:                              //min栈简化的基本运算算法，设为私有的
    bool minempty()                   //判断min栈是否空
    {
        return mintop==-1;
    }
    void minpush(T e)                 //元素e进min栈
    {   mintop++;
        mindata[mintop]=e;
    }
    T minpop()                        //元素出min栈
    {   T x=mindata[mintop];
        mintop--;
        return x;
    }
    T mingettop()                     //取min栈栈顶元素
    {
        return mindata[mintop];
    }
```

```
public:                                          //主栈基本运算算法，设为公有的
    bool empty()                                 //判断主栈是否空
    {
        return top==-1;
    }

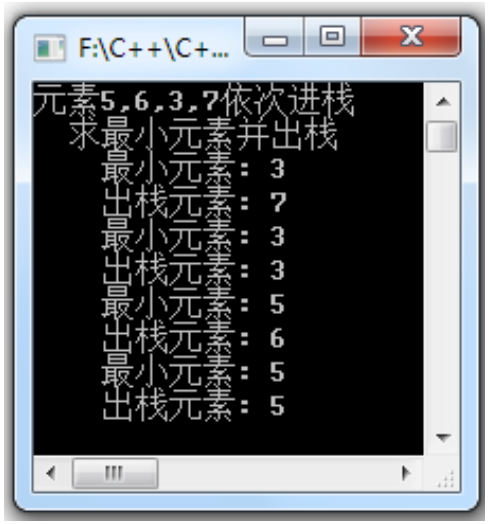
    bool push(T x)                               //元素x进主栈
    {   if (top==MaxSize-1)                      //主栈满返回false
            return false;
        if (empty() || x<=Getmin())
            minpush(x);                          //栈空或者x<=min栈顶元素时进min栈
        top++;
        data[top]=x;                             //将x进主栈
        return true;
    }
```

```
    bool pop(T& x)                  //元素x出主栈
    {  if (empty())                 //栈为空的情况，即栈下溢出
          return false;
       x=data[top];                 //从主栈出栈x
       top--;
       if (x==mingettop())          //若栈顶元素为最小元素
          minpop();                 //min栈出栈一次
       return true;
    }

    bool gettop(T& e)               //取主栈栈顶元素
    {  if (empty())                 //栈为空的情况，即栈下溢出
          return false;
       e=data[top];                 //取栈顶指针位置的元素
       return true;
    }

    T Getmin()                      //获取栈中最小元素
    {
       return mingettop();          //返回min栈的栈顶元素即主栈中最小元素
    }
};
```

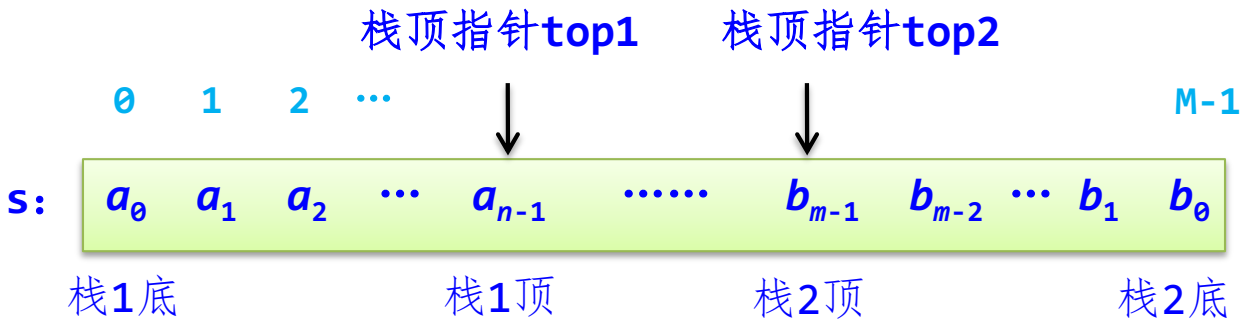
```
int main()
{   STACK<int> st;      //定义栈对象
    int e;
    cout <<"元素5,6,3,7依次进栈" << endl;
    st.push(5);
    st.push(6);
    st.push(3);
    st.push(7);
    cout << "  求最小元素并出栈" << endl;
    while (!st.empty())
    {   cout << "    最小元素: "
         << st.Getmin() << endl;
        st.pop(e);
        cout << "    出栈元素: " << e << endl;
    }
    return 0;
}
```

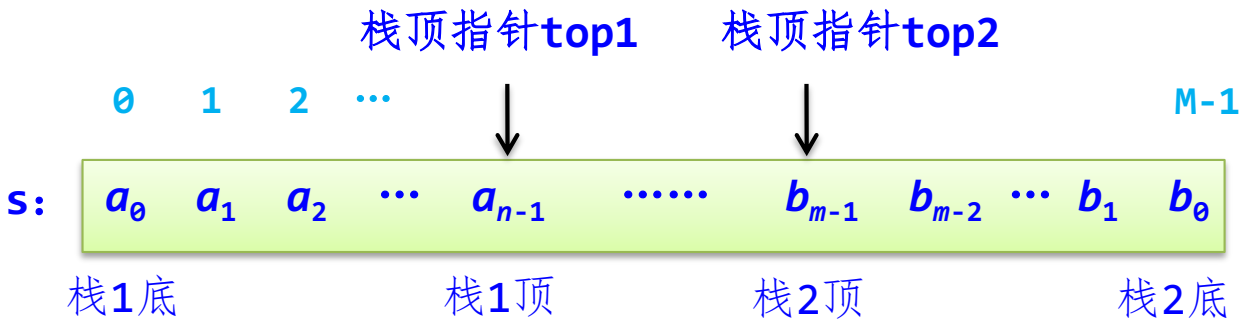


共享栈问题

【例3.7】设有两个栈S1和S2，它们都采用顺序栈存储，并且共享一个固定容量的存储区s[0..M-1]，为了尽量利用空间，减少溢出的可能，请设计这两个栈的存储方式。

解：为了尽量利用空间，减少溢出的可能，可以让两个的栈顶相向即进栈元素迎面增长的存储方式，为此设置两个栈的栈顶指针分别为top1和top2（均指向对应栈的栈顶元素）。



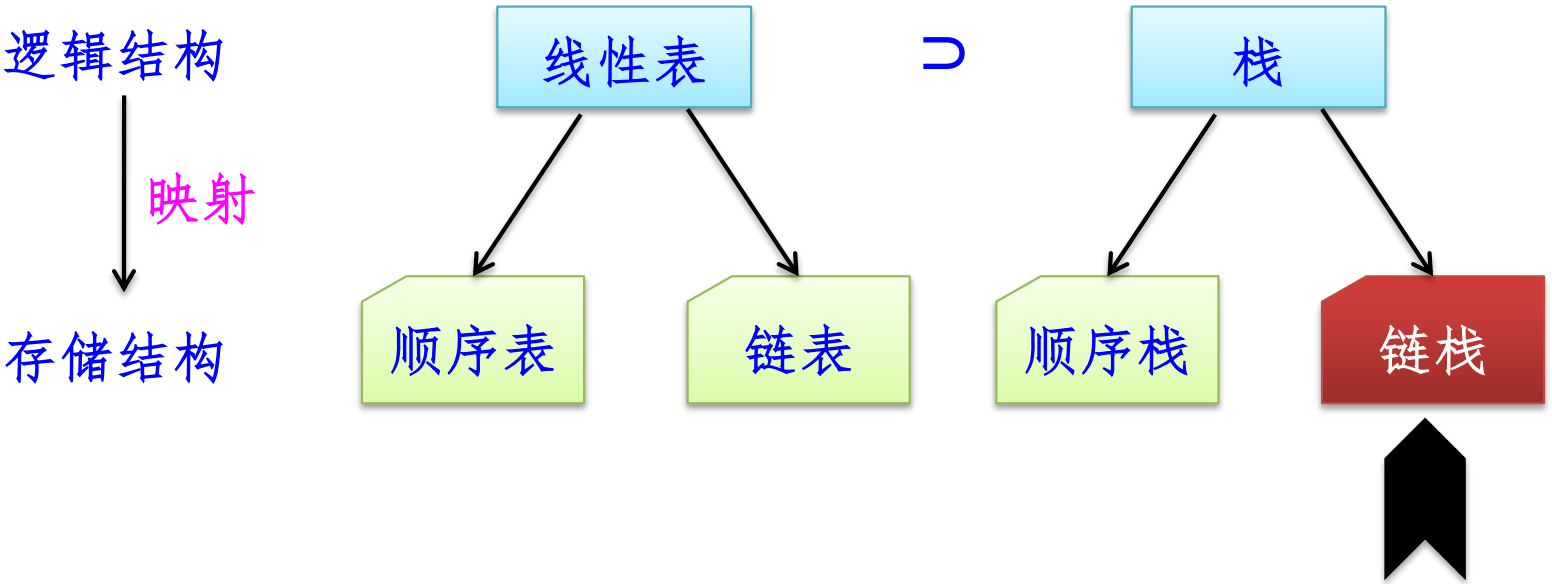


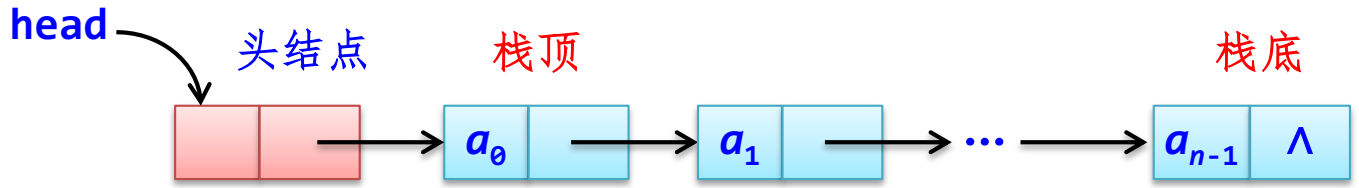
- 栈S1空的条件是top1=-1。
- 栈S1满的条件是top1=top2-1;
- 元素e进栈S1（栈不满时）的操作是：top1++;s[top1]=e。
- 元素e出栈S1（栈不空时）的操作是：e=s[top1];top1--。

- 栈S2空的条件是top2=M。
- 栈S2满的条件是top2=top1+1。
- 元素e进栈S2（栈不满时）的操作是：top2--;s[top2]=e。
- 元素e出栈S2（栈不空时）的操作是：e=s[top2];top2++。

3.1.4 栈的链式存储结构及其基本运算算法实现

栈的实现方式





初始时只含有一个头结点head并置head->next为NULL。这样链栈的四要素如下：

- 栈空的条件：head->next==NULL。
- 由于只有在内存溢出才会出现栈满，通常不考虑这种情况。
- 元素e进栈操作：将包含该元素的结点s插入作为首结点。
- 出栈操作：返回首结点值并且删除该结点。

和单链表一样，链栈中每个结点的类型LinkNode如下

```
template <typename T>
struct LinkNode                                  //链栈结点类型
{  T data;                                       //数据域
   LinkNode* next;                               //指针域
   LinkNode():next(NULL) {}                      //构造函数
   LinkNode(T d):data(d),next(NULL) {}           //重载构造函数
};
```

链栈类模板LinkStack

```
template <typename T>
class LinkStack                    //链栈类模板
{
public:
    LinkNode<T>* head;             //链栈头结点
    #栈的基本运算算法
};
```

（1）链栈的初始化和销毁

（2）判断栈是否为空empty()

（3）进栈push(e)

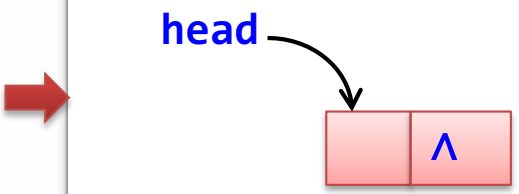
（4）出栈pop()

（5）取栈顶元素gettop()

链栈的基本运算算法

（1）链栈的初始化和销毁

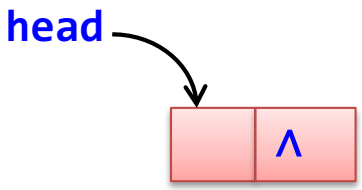
```
LinkStack()                    //构造函数
{
    head=new LinkNode<T>();
}
```




```
~LinkStack()                   //析构函数
{   LinkNode<T>* pre=head,*p=pre->next;
    while (p!=NULL)
    {   delete pre;
        pre=p; p=p->next;          //pre、p同步后移
    }
    delete pre;
}
```

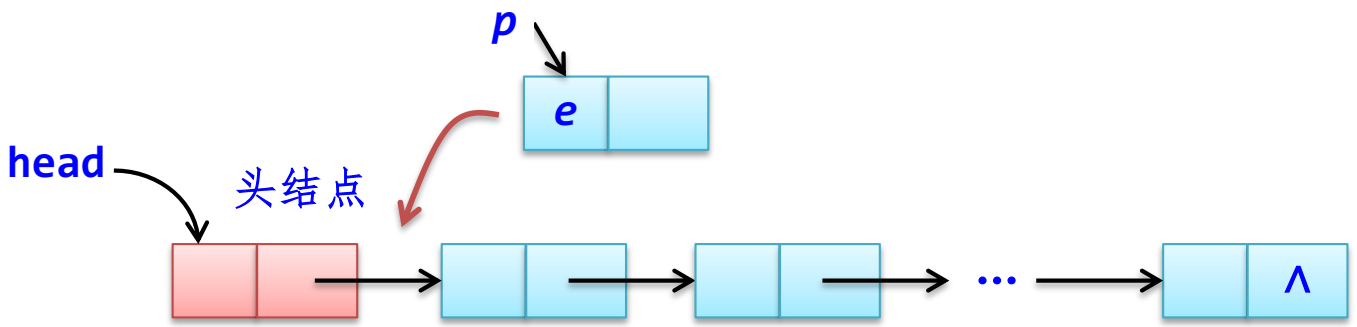
（2）判断栈是否为空empty()

```
bool empty()                          //判断栈是否为空
{
    return head->next==NULL;
}
```



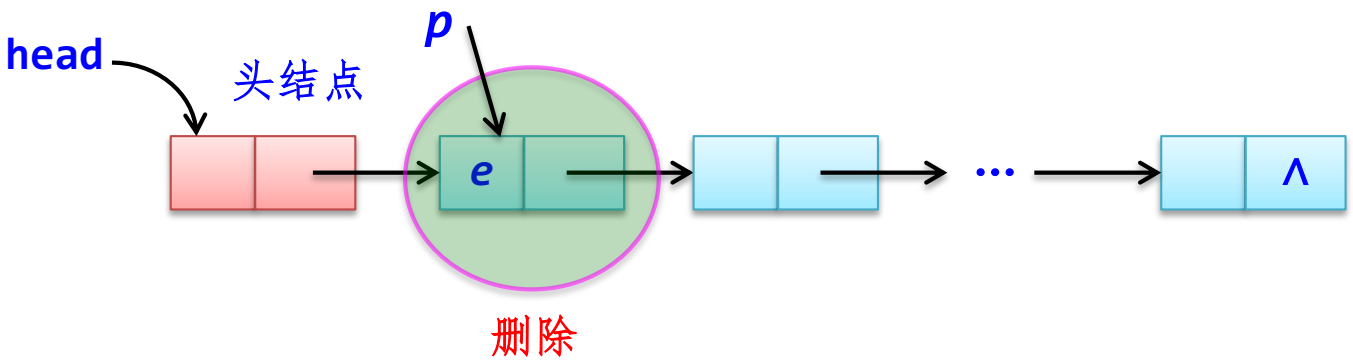
（3）进栈push(e)

```
bool push(T e)                                  //进栈算法
{   LinkNode<T>* p=new LinkNode<T>(e);          //新建结点p
    p->next=head->next;                         //插入结点p作为首结点
    head->next=p;
    return true;
}
```



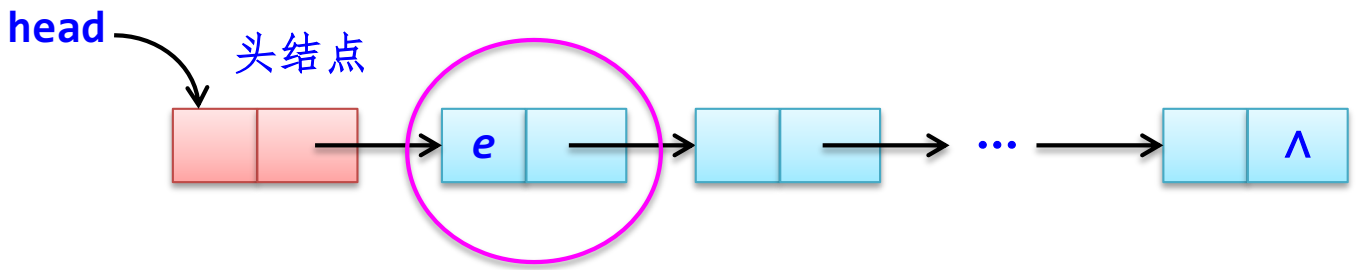
（4）出栈pop()

```
bool pop(T& e)                          //出栈算法
{  LinkNode<T>* p;
   if (head->next==NULL)                //栈空的情况
      return false;
   p=head->next;                        //p指向开始结点
   e=p->data;
   head->next=p->next;                  //删除结点p
   delete p;                            //释放结点p
   return true;
}
```



（5）取栈顶元素gettop()

```
bool gettop(T& e)                          //取栈顶元素
{  LinkNode<T>* p;
   if (head->next==NULL)                   //栈空的情况
      return false;
   p=head->next;                           //p指向开始结点
   e=p->data;
   return true;
}
```



链栈的几个问题



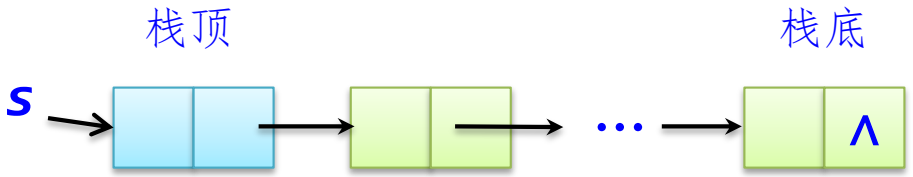
问题1：在以下几种存储结构中，哪个最适合用作链栈？

（1）带头结点的单链表

（2）不带头结点的循环单链表

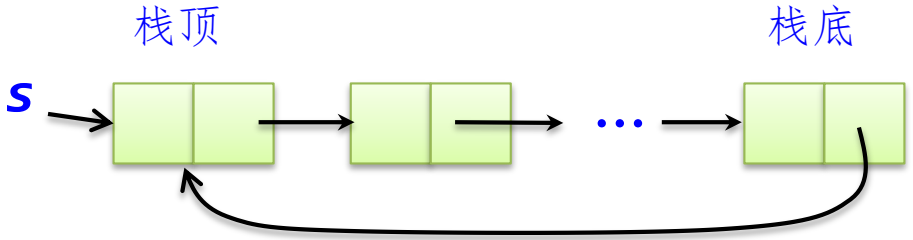
（3）带头结点的双链表

（1）



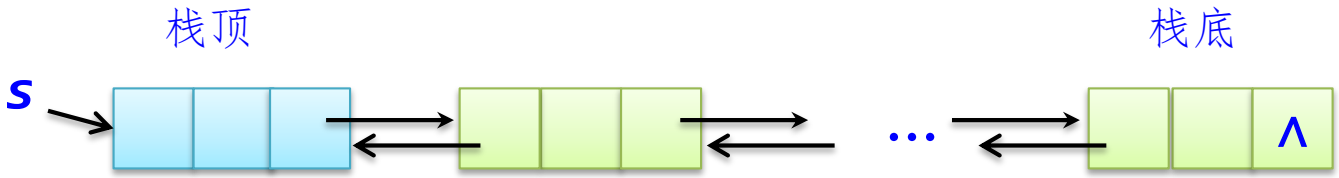
push和pop均为O(1)

（2）



push和pop均为O(*n*)

（3）



push和pop均为O(1)

（1）最好！

问题2：在一个算法中需要建立多个栈时可以选用以下三种方案之一，试问这三种方案之间相比各有什么优缺点？

（1）分别用多个顺序存储空间建立多个独立的顺序栈。

（2）多个栈共享一个顺序存储空间。

（3）分别建立多个独立的链栈。

答：（1）优点是每个栈仅用一个顺序存储空间时，操作简单。缺点是分配空间小了，容易产生溢出，分配空间大了，容易造成浪费，各栈不能共享空间。

（2）优点是多个栈仅用一个顺序存储空间，充分利用了存储空间，只有在整个存储空间都用完时才会产生溢出。缺点是当栈个数大于等于3时其中一个栈满时需要向左、右查询有无空闲单元的过程复杂且十分耗时。

（3）优点是多个链栈一般不考虑栈的溢出，采用动态空间分配具有良好的适应性。缺点是栈中元素要以指针相链接，比顺序存储多占用了存储空间。

3.1.5 链栈的应用算法设计示例

【例3.8】设计一个算法利用栈的基本运算将一个整数链栈中所有元素逆置。例如链栈st中元素从栈底到栈顶为（1，2，3，4），逆置后为（4，3，2，1）。



这里要求利用栈的基本运算来设计算法，所以不能直接采用单链表逆置方法。先出栈st中所有元素并保存在一个数组a中，再将数组a中所有元素依次进栈。

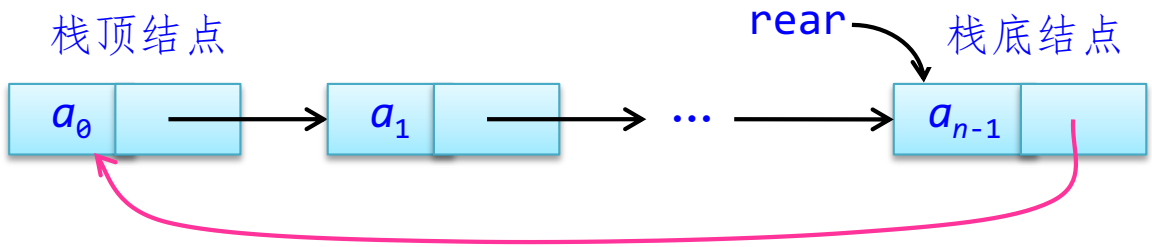
```
#include "LinkStack.cpp"                    //包含链栈类模板的定义
void Reverse(LinkStack<int>& st)            //逆置栈st
{  int a[MaxSize];                          //定义一个辅助数组
   int i=0,e;
   while (!st.empty())                      //将出栈的元素放到数组a中
   {  st.pop(e);
      a[i++]=e;
   }

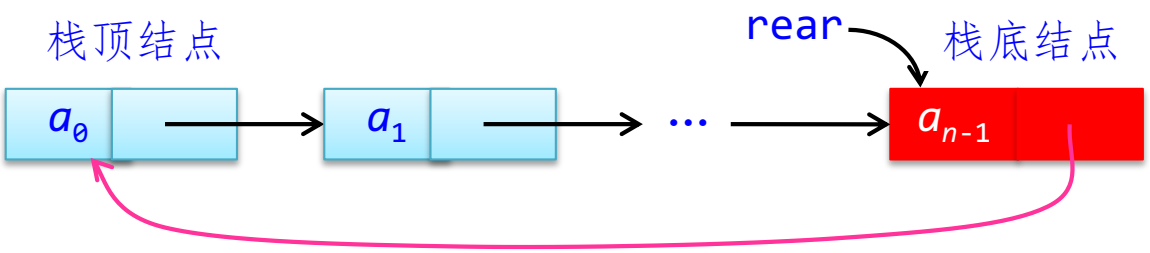
   for (int j=0;j<i;j++)                    //将数组a的所有元素进栈
      st.push(a[j]);
}
```

【例3.9】定义一个栈数据结构STACK，添加一个Getbottom()运算用于直接返回栈底元素（假设栈不空时）。要求采用链表实现并且函数Getbottom()、push以及pop的时间复杂度都是O(1)。



- 如果采用普通单链表实现，以前端为栈顶后端为栈底，那么找到尾结点（存放栈底元素）的时间复杂度为O(*n*)，不满足题目要求。
- 改为不带头结点仅有尾结点指针的循环单链表rear作为链栈。





初始时rear=NULL，栈的四要素如下：

① 栈空条件：rear=NULL。

② 栈满条件：不考虑。

③ 元素*e*进栈操作：建立含*e*元素的结点*p*，将结点*p*插入到rear结点的后面。

④ 元素*e*出栈操作：取结点rear之后的结点值*e*，删除该结点。

Getbottom()函数就是返回rear结点值即rear->data（栈不空时）

```
template <typename T>
struct LinkNode                                  //链栈结点类型
{   T data;                                      //数据域
    LinkNode* next;                              //指针域

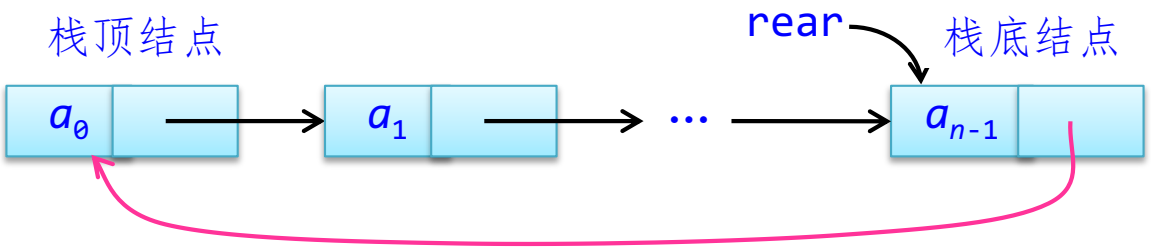
    LinkNode():next(NULL) {}                     //构造函数
    LinkNode(T d):data(d),next(NULL) {}          //重载构造函数
};
```

```
template <typename T>
class STACK                                          //链栈类模板
{
public:
   LinkNode<T>* rear;                                //链栈尾结点指针

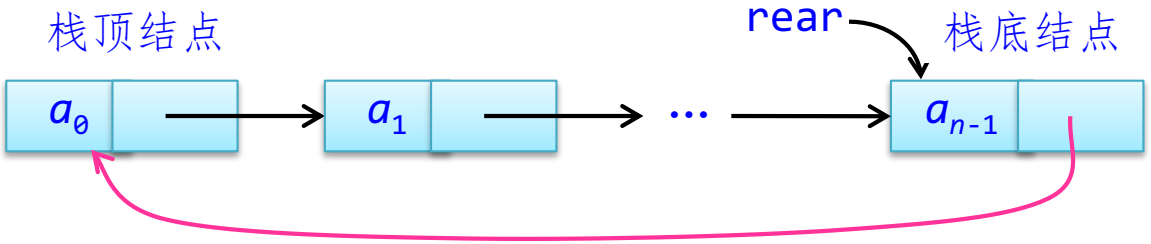
   STACK():rear(NULL) {}                             //构造函数

   ~STACK()                                          //析构函数
   {  if (rear==NULL) return;                        //空链表直接返回
      LinkNode<T>* pre=rear,*p=pre->next;
      while (p!=rear)
      {  delete pre;
         pre=p; p=p->next;                           //pre、p同步后移
      }
      delete pre;
   }

   bool empty()                                      //判栈空算法
   {
      return rear==NULL;
   }
```



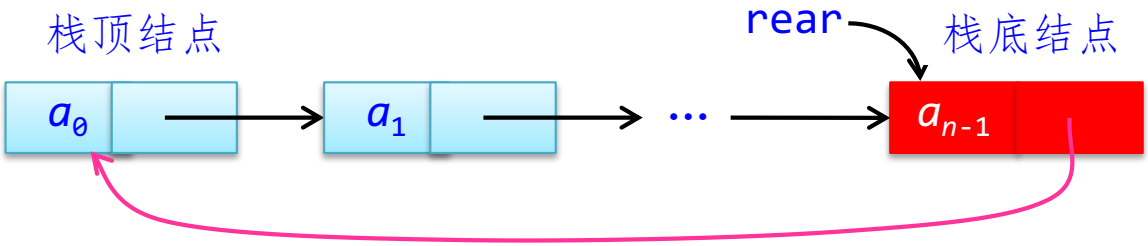

```
bool push(T e)                                  //进栈算法
{  LinkNode<T>* p=new LinkNode<T>(e); //新建结点p
   if (empty())                                 //栈为空的情况
   {  rear=p;
      rear->next=rear;
   }
   else                                         //栈不空的情况
   {  p->next=rear->next;                       //将结点p插入到结点rear之后
      rear->next=p;
   }
   return true;
}
```




```
bool pop(T& e)                          //出栈算法
{  LinkNode<T>* p;
   if (empty()) return false;           //栈空的情况
   if (rear->next==rear)                //栈中只有一个结点的情况
   {  p=rear;
      rear=NULL;
   }
   else                                 //栈中有2个及以上结点的情况
   {  p=rear->next;
      rear->next=p->next;
   }
   e=p->data;
   delete p;                            //释放结点p
   return true;
}
```

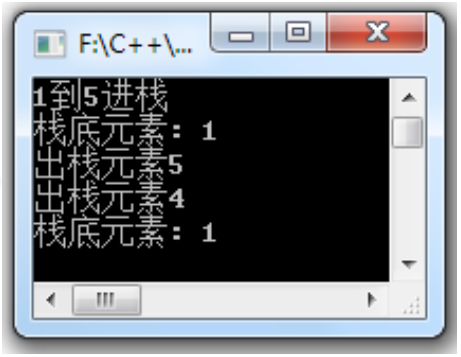
```
    bool gettop(T& e)                          //取栈顶元素
    {  if (empty()) return false;              //栈空的情况
       e=rear->next->data;
       return true;
    }

    T Getbottom()                              //取栈底元素
    {
       return rear->data;
    }
};
```



程序验证

```
int main()
{   int e;
    STACK<int> st;
    cout << "1到5进栈" << endl;
    for (int i=1;i<=5;i++)
        st.push(i);
    cout << "栈底元素: " << st.Getbottom() << endl;
    st.pop(e);
    cout << "出栈元素" <<  e << endl;
    st.pop(e);
    cout << "出栈元素" <<  e << endl;
    cout << "栈底元素: " << st.Getbottom() << endl;
    return 0;
}
```



2.6 STL中的线性表

- STL中有一类容量称为顺序容器，顺序容器按照线性次序的位置存储数据，即第1个元素，第2个元素，依此类推，简单地说，可以采用这些容量存放线性表。
- STL中的顺序容器有、string、deque和list。这里主要介绍vector和list。



自学，上机编程

C++序列式容器（STL序列式容器）是什么

< 上一节 下一节 >

所谓序列容器，即以线性排列（类似普通数组的存储方式）来存储某一指定类型（例如 int、double 等）的数据，需要特殊说明的是，该类容器并不会自动对存储的元素按照值的大小进行排序。

需要注意的是，序列容器只是一类容器的统称，并不指具体的某个容器，序列容器大致包含以下几类容器：

- array<T,N>（数组容器）：表示可以存储 N 个 T 类型的元素，是 C++ 本身提供的一种容器。此类容器一旦建立，其长度就是固定不变的，这意味着不能增加或删除元素，只能改变某个元素的值；
- vector<T>（向量容器）：用来存放 T 类型的元素，是一个长度可变的序列容器，即在存储空间不足时，会自动申请更多的内存。使用此容器，在尾部增加或删除元素的效率最高（时间复杂度为 O(1) 常数阶），在其它位置插入或删除元素效率较差（时间复杂度为 O(n) 线性阶，其中 n 为容器中元素的个数）；
- deque<T>（双端队列容器）：和 vector 非常相似，区别在于使用该容器不仅尾部插入和删除元素高效，在头部插入或删除元素也同样高效，时间复杂度都是 O(1) 常数阶，但是在容器中某一位置处插入或删除元素，时间复杂度为 O(n) 线性阶；
- list<T>（链表容器）：是一个长度可变的、由 T 类型元素组成的序列，它以双向链表的形式组织元素，在这个序列的任何地方都可以高效地增加或删除元素（时间复杂度都为常数阶 O(1)），但访问容器中任意元素的速度要比前三种容器慢，这是因为 list<T> 必须从第一个元素或最后一个元素开始访问，需要沿着链表移动，直到到达想要的元素。
- forward_list<T>（正向链表容器）：和 list 容器非常类似，只不过它以单链表的形式组织元素，它内部的元素只能从第一个元素开始访问，是一类比链表容器快、更节省内存的容器。

> 注意，其实除此之外，stack<T> 和 queue<T> 本质上也属于序列容器，只不过它们都是在 deque 容器的基础上改头换面而成，通常更习惯称它们为容器适配器，有关它们的介绍，会放到后续章节中。

3.1.6 STL中的栈容器

- STL中的stack栈容器具有后进先出的特点，只有一个进出口即栈顶，可以在栈顶插入（进栈）和删除（出栈）元素，而不允许像数组那样从前向后或者从后向前顺序遍历。
- stack是一种适配器容器，即使用一个特定容器类的封装对象作为它的底层容器，简单地说，stack的数据存放在底层容器中，并且利用底层容器提供的成员函数如back()、push_back()、pop_back()实现stack的功能。
- 如果未特别指定stack的底层容器，默认使用双端队列deque容器作为底层容器。也可以指定vector或者list作为底层容器。

以下语句用于定义4个stack对象

```
stack<int> st1;                    //定义一个整数栈st1
stack<int> st2(st1);               //由st1栈复制产生st2栈
stack<int, vector<int>> st3;       //定义整数栈st3，以vector作为底层容器
stack<int, list<int>> st4;         //定义整数栈st4，以list作为底层容器
```

stack容器主要的成员函数

成员函数	说明
empty()	判断栈是否为空
size()	返回栈中的实际元素个数
push(e)	即元素e进栈
top()	返回栈顶元素，当栈空时抛出异常
pop()	出栈一个元素，并不返回出栈的元素，当栈空时抛出异常

提示

- 与前面自己实现的栈运算相比，stack容器的成员函数使用更加简单方便。
- 需要注意的是stack容器具有空间动态扩展功能，push()不会出现上溢出的情况，另外在使用top()和pop()之前应保证栈不空。

```
#include<iostream>
#include <stack>
using namespace std;
int main()
{   stack<int> st;
    st.push(1); st.push(2); st.push(3);
    printf("栈顶元素: %d\n",st.top());
    printf("出栈顺序: ");
    while (!st.empty())                    //栈不空时出栈所有元素
    {   printf("%d ",st.top());
        st.pop() ;
    }
    printf("\n");
    return 0;
}
```



```
栈顶元素: 3
出栈顺序: 3 2 1
```

3.1.7 栈的综合应用

求解问题中需要临时保存一些数据元素：

- 先保存的后处理：栈
- 先保存的先处理：队列

常用来处理具有递归结构的数据

- 深度优先搜索
- 数制转换
- 表达式求值
- 行编辑处理
- 子程序 / 函数调用的管理
- 消除递归

数制转换

- 非负十进制数转换成其它进制的一种方法

✓ 例，十进制转换成八进制：$(66)_{10}=(102)_8$

- 66/8 = 8 余 2
- 8/8 = 1 余 0
- 1/8 = 0 余 1

结果为余数的逆序：102

- 特点：

最先求得的余数在输出结果中最后写出，最后求出的余数最先写出，符合栈的先入后出性质，故可用栈来实现数制转换

✓ 同理，一个非负十进制整数 N 转换为另一个等价的B进制数，可以采用“除B取余法”来解决 。

简单表达式求值

exp：仅包含“+”、“-”、“*”、“/”、正整数和小括号的合法数学表达式–**中缀表达式**。

后缀表达式：就是运算符在操作数的后面，已经考虑了运算符的优先级，不包含括号，只含操作数和运算符。

1+2*3



123*+

运算算法设计

运算符位于两个操作数中间的表达式称为中缀表达式。

例如，1+2*3就是一个中缀表达式。

中缀表达式的运算规则：“先乘除，后加减，从左到右计算，先括号内，后括号外”。

因此，中缀表达式不仅要依赖运算符优先级，而且还要处理括号。

算术表达式的另一种形式是后缀表达式或逆波兰表达式，就是在算术表达式中，运算符在操作数的后面，如1+2*3的后缀表达式为1 2 3 * +。

后缀表达式：

- 已考虑了运算符的优先级
- 没有括号
- 只有操作数和运算符，而且越放在前面的运算符来越优先执行

在算术表达式中，如果运算符在操作数的前面，称为前缀表达式，如1+2*3的前缀表达式为+ 1 * 2 3。

- 中缀表达式：1 + 2 * 3
- 后缀表达式：1 2 3 * +
- 前缀表达式：+ 1 * 2 3

运算数的相对次序相同

exp求值过程

- 将表达式exp转换成后缀表达式postexp。
- 对该后缀表达式求值。

设计求表达式值的类Express

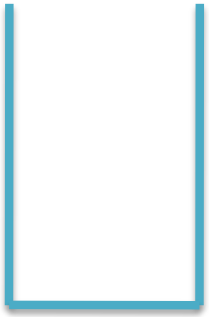
```
class Express                                   //求表达式值类
{  string exp;                                  //存放中缀表达式
   string postexp;                              //存放后缀表达式
public:
   Express(string str)                          //构造函数
   {  exp=str;
      postexp="";
   }
   string getpostexp()                          //返回postexp
   {
      return postexp;
   }
   void Trans()        { … }                    //将exp转换成postexp
   double GetValue() { … }                      //计算后缀表达式postexp的值
};
```

（1）将算术表达式转换成后缀表达式

exp ⇒ *postexp*

扫描*exp*的所有字符：

- **数字字符直接放在*postexp*中**
- **运算符通过一个栈来处理优先级**



运算符栈

中缀：1 + 2 * 3

后缀：1 2 3 * +

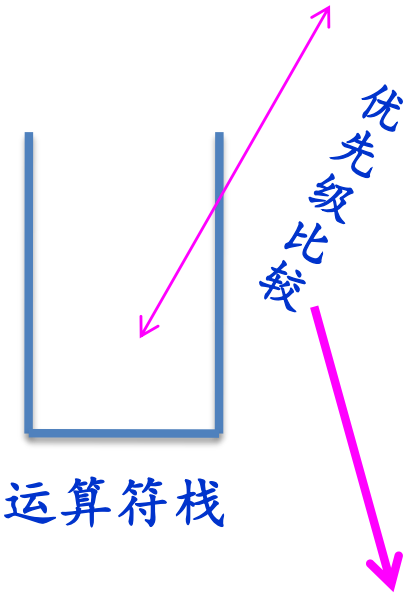
中缀：2*(1+3)-4

后缀：2 1 3 + * 4 -

exp ⇒ *postexp*

情况1（没有括号）

exp= “1 + 2 + 3”



postexp：

“1+2+3” ⇒ “1 2 + 3 +”

- 先进栈的先退栈即先执行：

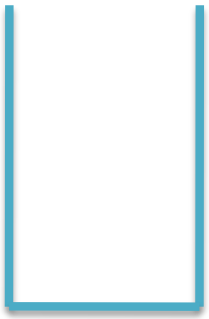
  只有大于栈顶优先级才能直接进栈

- *exp*扫描完毕，所有运算符退栈

exp ⇒ postexp

情况2（带有括号）

*exp=“2 * (1 + 3) - 4 ”*



运算符栈

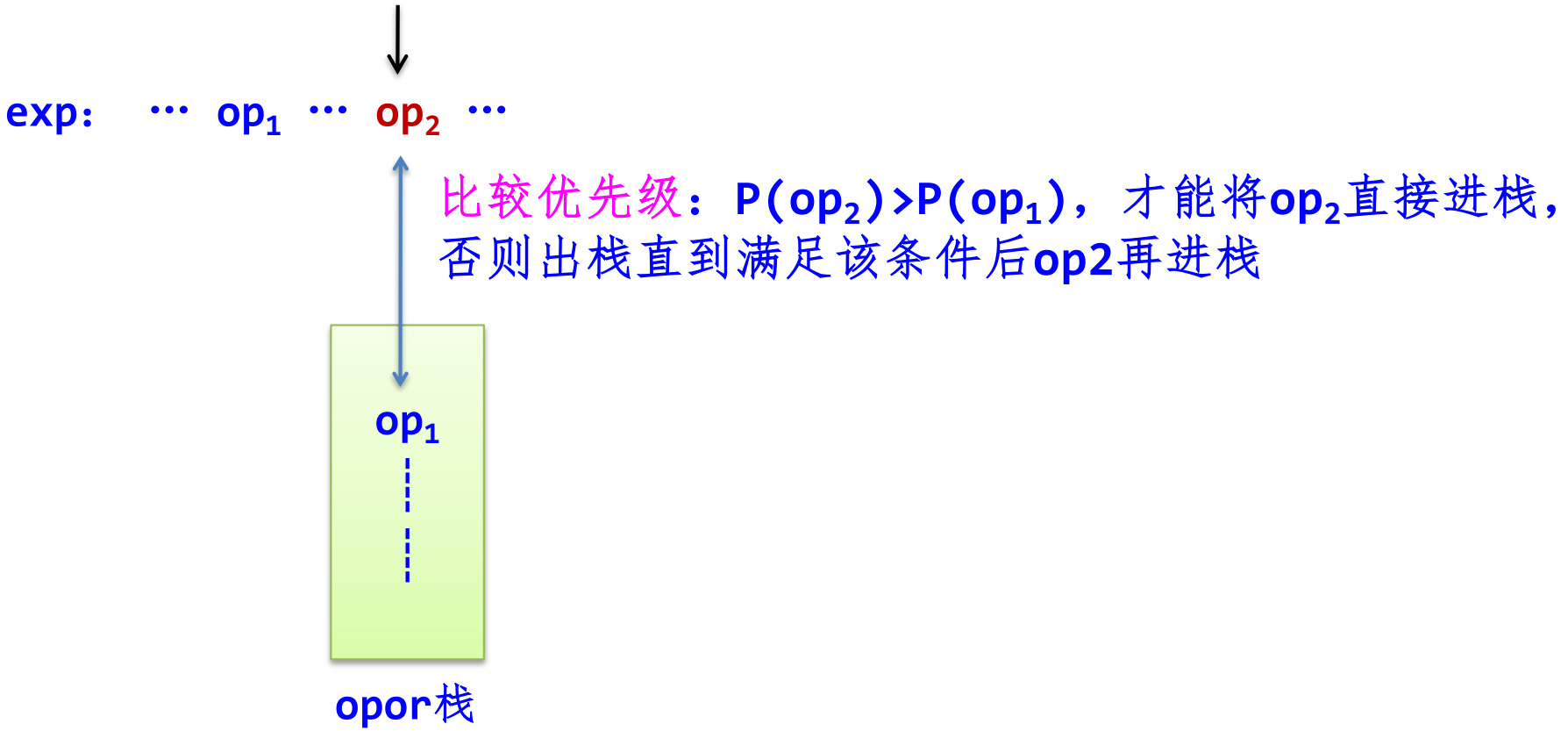
postexp：

“2*(1+3)-4” ⇒ “2 1 3 + * 4 -”

- 开始时，任何运算符都进栈
- (：一个子表达式开始，进栈
- 栈顶为(：任何运算符进栈
-)：退栈到(
- 只有大于栈顶的优先级，才进栈；否则退栈

exp ⇨ postexp

使用运算符栈opor：先出栈的运算符先做运算！



括号的特殊性！

转换过程

while (若exp未读完)

{ 从exp读取字符ch;

ch为数字：将后续的所有数字均依次存放到postexp中;

ch为左括号'('：将'('进栈到opor;

ch为右括号')'：将opor栈中'('以前的运算符依次出栈并存放到postexp中,再将'('退栈;

若ch的优先级高于栈顶运算符优先级，则将ch进栈；否则出栈并存放到postexp中直到该条件成立,再将ch进oper栈;

}

字符串exp扫描完毕,则退栈opor的所有运算符并存放到postexp中

表达式“(56-20)/(4+2)”转换成后缀表达式的过程

ch	操 作	postexp	opor栈
(			
56			
-			
20			
)			
/			
(			
4			
+			
2			
)			

表达式“(56-20)/(4+2)”转换成后缀表达式的过程

ch	操 作	postexp	opor栈
(	将'('进opor栈		(
56	将56存入postexp中，并插入一个字符'#'	"56#"	(
-	由于opor中'('以前没有字符，则直接将'-'进opor栈	"56#"	(-
20	将20#存入postexp中	"56#20#"	(-
)	将栈opor中'('以前的运算符依次出栈并存入postexp，然后将'('出栈	"56#20#-"	
/	将'/'进opor栈	"56#20#-"	/
(	将'('进opor栈	"56#20#-"	/(
4	将4#存入postexp	"56#20#-4#"	/(
+	由于opor中'('以前没有运算符，则直接将'+'进opor栈	"56#20#-4#"	/(+
2	将2#存入postexp	"56#20#-4#2#"	/(+
)	将opor栈中'('以前的运算符依次出栈并存入postexp，然后将'('出栈	"56#20#-4#2#+"	/
	exp扫描完毕，将opor栈中所有运算符出栈并存入postexp，得到最后的后缀表达式	"56#20#-4#2#+/"	

```
void Trans()                        //将算术表达式exp转换成后缀表达式postexp
{  stack<char> opor;                //定义运算符栈opor
   int i=0;                         //i为exp的下标
   char ch,e;
   while (i<exp.length())           //exp表达式未扫描完时循环
   {  ch=exp[i];
      if (ch=='(')                  //遇到左括号
         opor.push(ch);             //将左括号直接进栈
      else if (ch==')')             //遇到右括号
      {  while (!opor.empty() && opor.top()!='(')
         {  e=opor.top();           //将栈中'('之前的运算符退栈并存入postexp
            opor.pop();
            postexp+=e;
         }
         opor.pop();                //将(退栈
      }
```

```
else if (ch=='+' || ch=='-')        //遇到加或减号
{  while (!opor.empty() && opor.top()!='(')
   {  e=opor.top();                 //将栈中(之前的所有运算符退栈
      opor.pop();                   //并存入postexp
      postexp+=e;
   }
   opor.push(ch);                   //再将'+'或'-'进栈
}
else if (ch=='*' || ch=='/')        //遇到'*'或'/'号
{  while (!opor.empty() && opor.top()!='(' &&
             (opor.top()=='*' || opor.top()=='/'))
   {  e=opor.top();                 //将栈中(之前的所有*或/依次出栈
      opor.pop();                   //并存入postexp
      postexp+=e;
   }
   opor.push(ch);                   //再将'*'或'/'进栈
}
```

```
    else                                    //遇到数字字符
    {  string d="";
       while (ch>='0' && ch<='9')           //遇到数字
       {  d+=ch;                            //提取所有连续的数字字符
          i++;
          if (i<exp.length())               //exp没有遍历完时取下一个字符ch
             ch=exp[i];
          else                              //exp遍历完毕时退出数字判断
             break;
       }
       i--;                                 //退一个字符
       postexp+=d;                          //将数字串存入postexp
       postexp+="#";                        //用#标识一个数字串结束
    }
    i++;                                    //继续处理其他字符
}
```

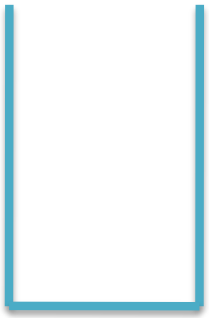
```
    while (!opor.empty())      //此时exp扫描完毕,栈不空时循环
    {  e=opor.top();
       opor.pop();             //将栈中所有运算符退栈并放入postexp
       postexp+=e;
    }
}
```

（2）后缀表达式求值

postexp ⇒ 值　　“2 1 3 + * 4 -” ⇒ ??

扫描*postexp*的所有字符：

- **数字字符：转换为数值并进栈**
- **运算符：退栈两个操作数，计算，将结果进栈**



操作数栈

后缀表达式postexp求值

使用运算数栈opand

```
while (若postexp未读完)
{   从postexp读取字符ch;
    ch为'+': 从opand栈出栈两个数值a和b,计算c=b+a;将c进栈opand;
    ch为'-': 从opand栈出栈两个数值a和b,计算c=b-a;将c进栈opand;
    ch为'*': 从opand栈出栈两个数值a和b,计算c=b*a;将c进栈opand;
    ch为'/': 从opand栈出栈两个数值a和b,若a不零,计算c=b/a;将c
        进栈opand;
    ch为数字字符: 将连续的数字串转换成数值d,将d进栈opand;
}
opand栈中唯一的数值即为表达式值
```

后缀表达式"56#20#-4#2#+/"的求值过程

ch序列	说明	st栈
56	遇到56，将56进栈	56
20	遇到20，将20进栈	56,20
'-'	遇到'-'，出栈两次，将56-20=36进栈	36
4	遇到4，将4进栈	36,4
2	遇到2，将2进栈	36,4,2
'+'	遇到'+'，出栈两次，将4+2=6进栈	36,6
'/'	遇到'/'，出栈两次，将36/6=6进栈	6
	postexp扫描完毕，算法结束，栈顶数值6即为所求	

```
double GetValue()                                   //计算后缀表达式postexp的值
{  stack<double> opand;                             //定义运算数栈opand
   double a,b,c,d;
   char ch;
   int i=0;
   while (i<postexp.length())                       //postexp字符串未扫描完时循环
   {  ch=postexp[i];
      switch (ch)
      {
      case '+':                                     //遇到+
         a=opand.top(); opand.pop();                //退栈运算数a
         b=opand.top(); opand.pop();                //退栈运算数b
         c=b+a;                                     //计算c
         opand.push(c);                             //将计算结果进栈
         break;
```

```
case '-':                                   //遇到-
    a=opand.top(); opand.pop();             //退栈运算数a
    b=opand.top(); opand.pop();             //退栈运算数b
    c=b-a;                                  //计算c
    opand.push(c);                          //将计算结果进栈
    break;

case '*':                                   //遇到*
    a=opand.top(); opand.pop();             //退栈运算数a
    b=opand.top(); opand.pop();             //退栈运算数b
    c=b*a;                                  //计算c
    opand.push(c);                          //将计算结果进栈
    break;

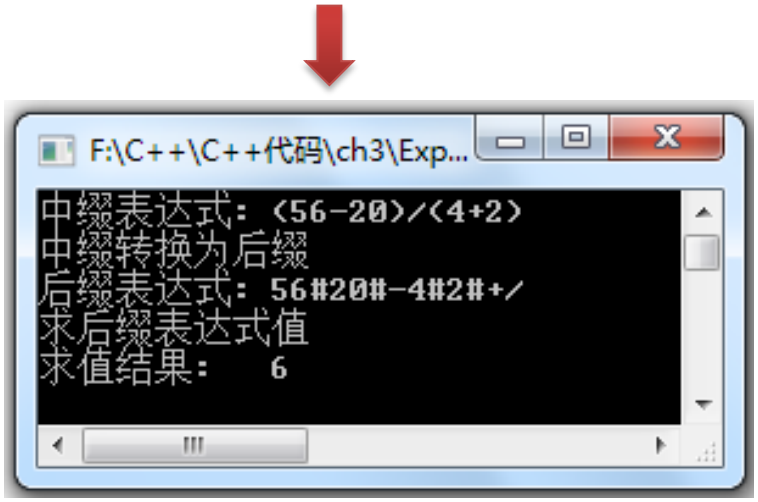
case '/':                                   //遇到/
    a=opand.top(); opand.pop();             //退栈运算数a
    b=opand.top(); opand.pop();             //退栈运算数b
    c=b/a;                                  //计算c
    opand.push(c);                          //将计算结果进栈
    break;
```

```
        default:                        //遇到数字字符
            d=0;                        //将连续的数字符转换成数值存放到d中
            while (ch>='0' && ch<='9')
            {  d=10*d+(ch-'0');
               i++;
               ch=postexp[i];
            }
            opand.push(d);              //将数值d进栈
            break;
        }
        i++;                            //继续处理其他字符
    }
    return opand.top();                 //栈顶元素即为求值结果
}
```

程序验证

主程序

```
int main()
{   string str="(56-20)/(4+2)";
    Express obj(str);
    cout << "中缀表达式: " << str << endl;
    cout << "中缀转换为后缀" << endl;
    obj.Trans();
    cout << "后缀表达式: " << obj.getpostexp() << endl;
    cout << "求后缀表达式值" << endl;
    cout << "求值结果:   " << obj.GetValue() << endl;
    return 0;
}
```



前面的过程可以进一步优化吗?

求表达式值的两个步骤可以合并起来，不必产生后缀表达式:

- 将所有遇到的运算数进opand栈。
- 在opor出栈一个运算符op时，从opand中依次出栈两个运算数*a*、*b*，执行*c* = *b* op *a*运算，将*c*进opand栈。

手工转换产生后缀表达式

5+2*(1+6)-8/2

加括号

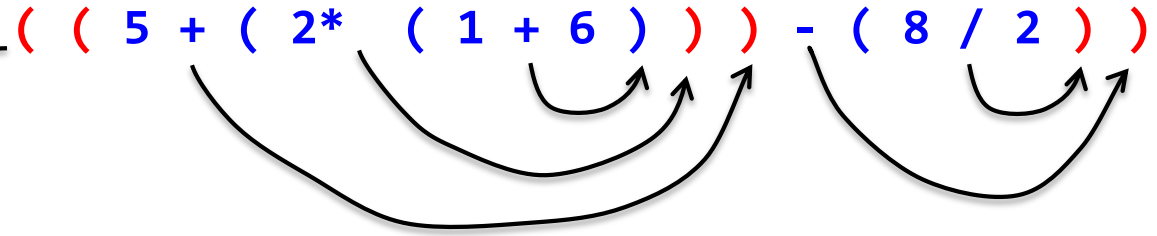
5+(2*(1+6))-8/2

5+(2*(1+6))-(8/2)

(5+(2*(1+6)))-(8/2)

((5+(2*(1+6)))-(8/2))

除括号



5 2 1 6 +* + 8 2 / -

示例

2018年全国硕士研究生入学统一考试题

1、若栈S_1中保存整数，栈S_2中保存运算符，函数F()依次执行下述各步操作：

（1）从S_1中依次弹出两个操作数*a*和*b*；

（2）从S_2中弹出一个运算符op；

（3）执行相应的运算*b* op *a*；

（4）将运算结果压入S_1中。

假定S_1中的操作数依次是5，8，3，2（2在栈顶），S_2中的运算符依次是*，-，+（+在栈顶）。调用3次F()后，S1栈顶保存的值是（　）。

A.-15　　B.15　　C.-20　　D.20

按求后缀表达式值的过程操作！



2014年全国硕士研究生入学统一考试题

2．假设栈初始为空，将中缀表达式*a/b+(c*d-e*f)/g*转换为等价的后缀表达式的过程中，　当扫描到*f*时，栈中的元素依次是（　）。

A．+ (* -　　B．+ (-*　　C．/ + (* -*　　D．/ + -*

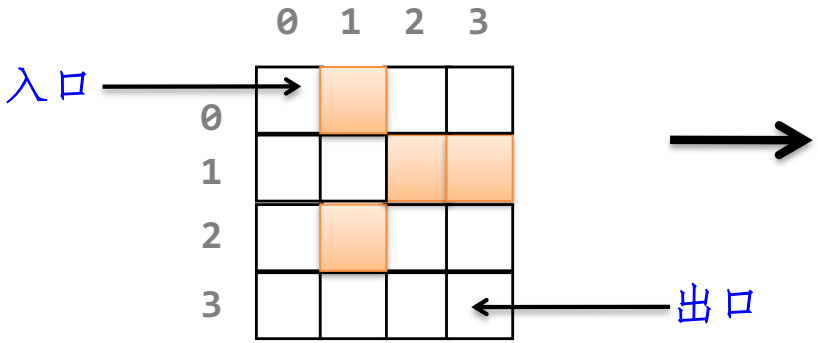
按中缀转换为后缀表达式的过程操作！

求迷宫问题

给定一个*M*×*N*的迷宫图、入口与出口、行走规则。求一条从指定入口到出口的路径。

所求路径必须是简单路径，即路径不重复。

求迷宫问题

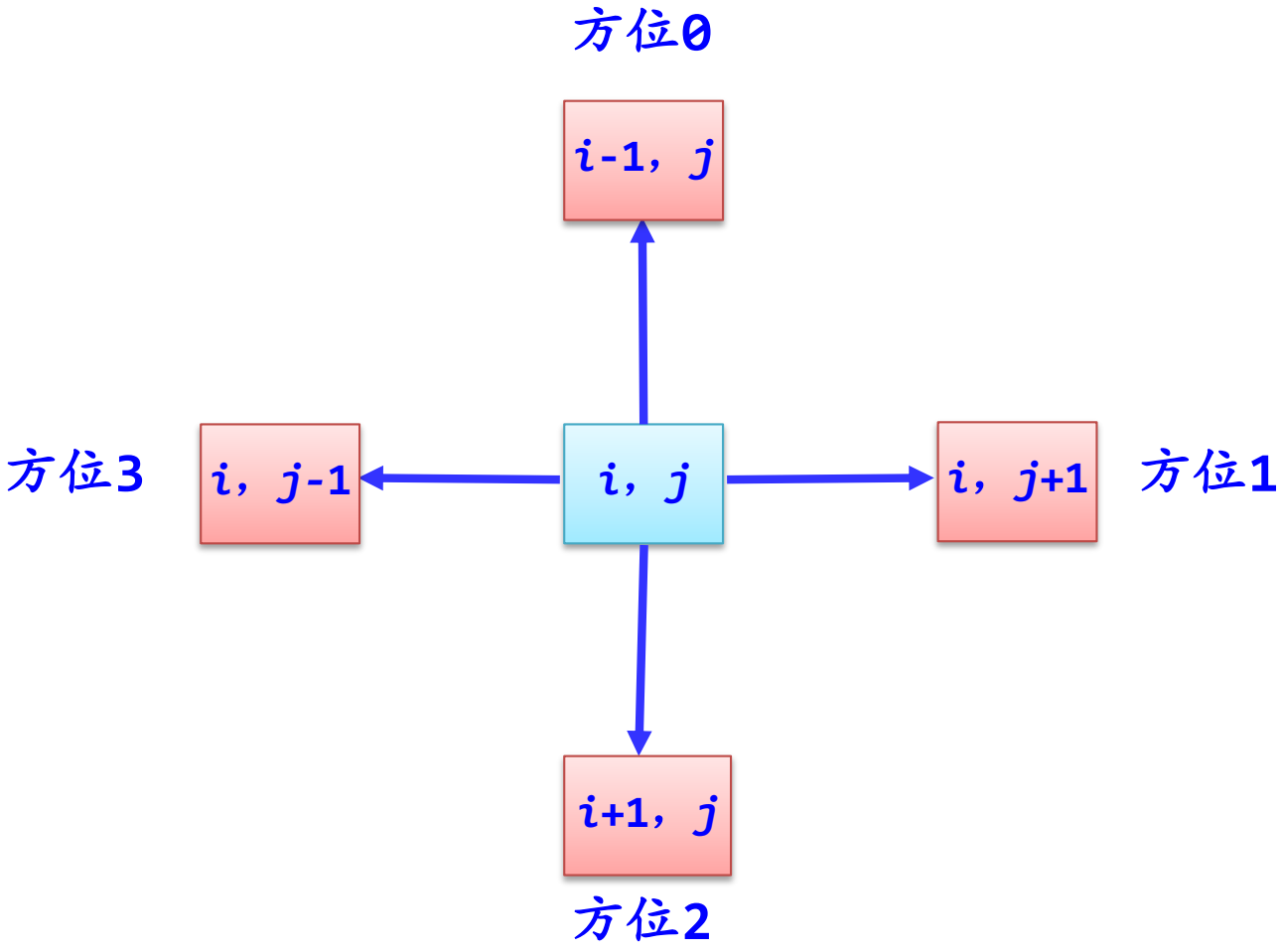



```
int mg[MAX][MAX]=
{{0,1,0,0},
 {0,0,1,1},
 {0,1,0,0},
 {0,0,0,0}
};
int m=4,n=4;
```

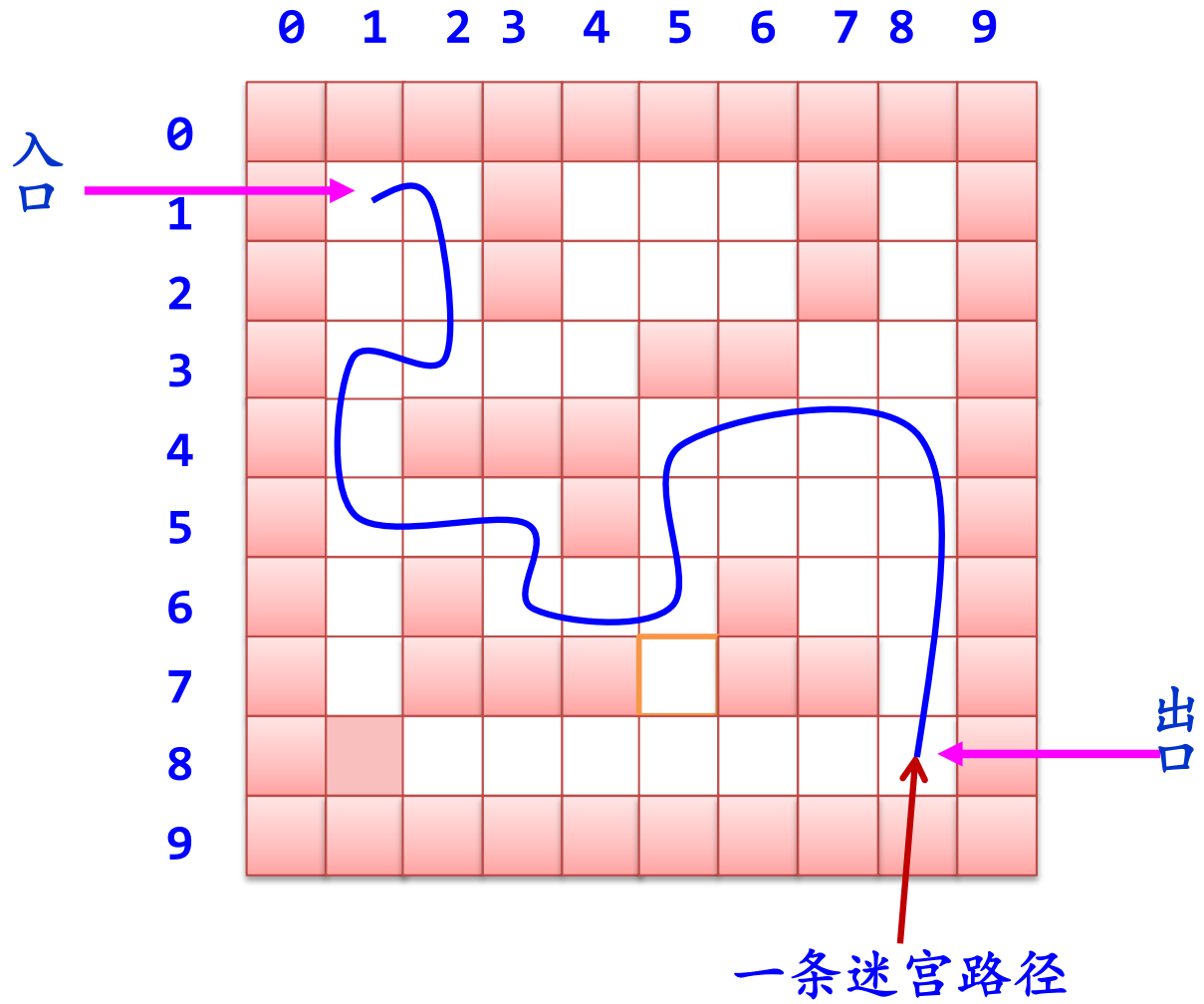
一个迷宫图

求从入口到出口的一条简单路径

行走规则：上、下、左、右相邻方块行走。其中（i，j）表示一个方块



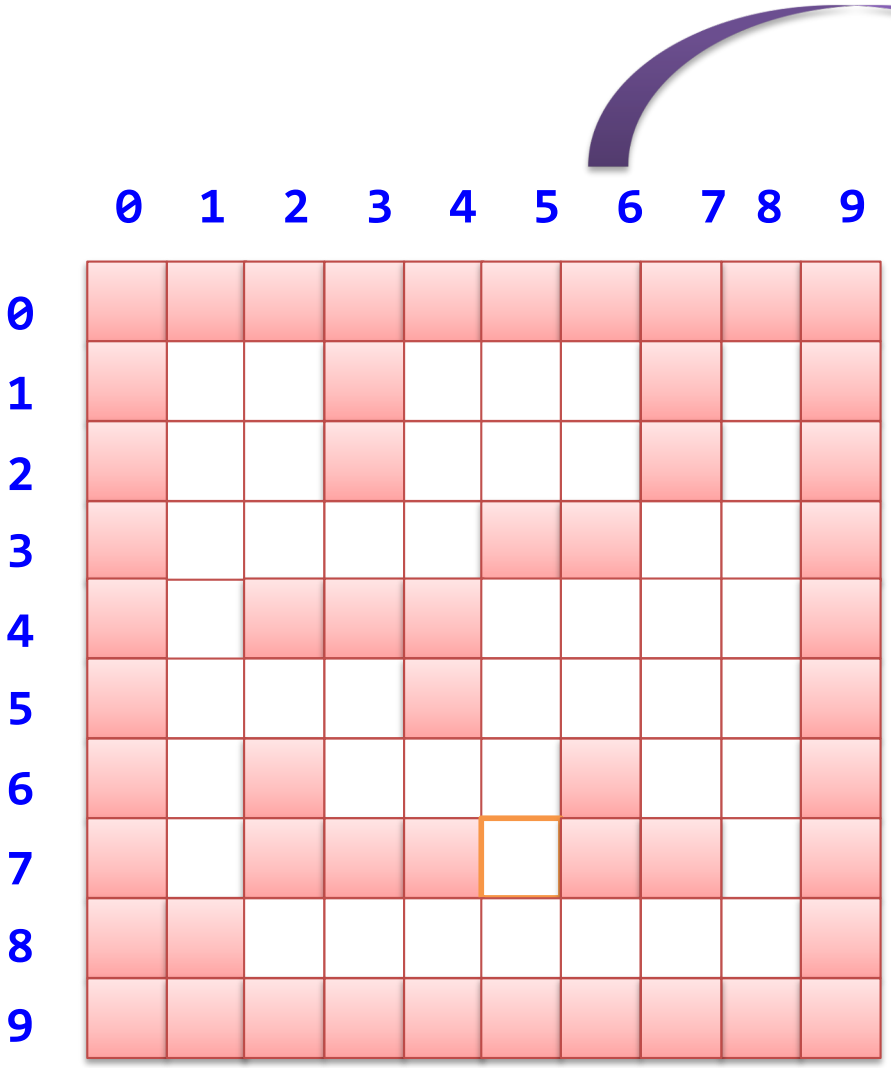
例如，*M*=8，*N*=8，图中的每个方块，用空白表示通道，用阴影表示障碍物。为了算法方便，一般在迷宫外围加上了一条围墙。





数据组织

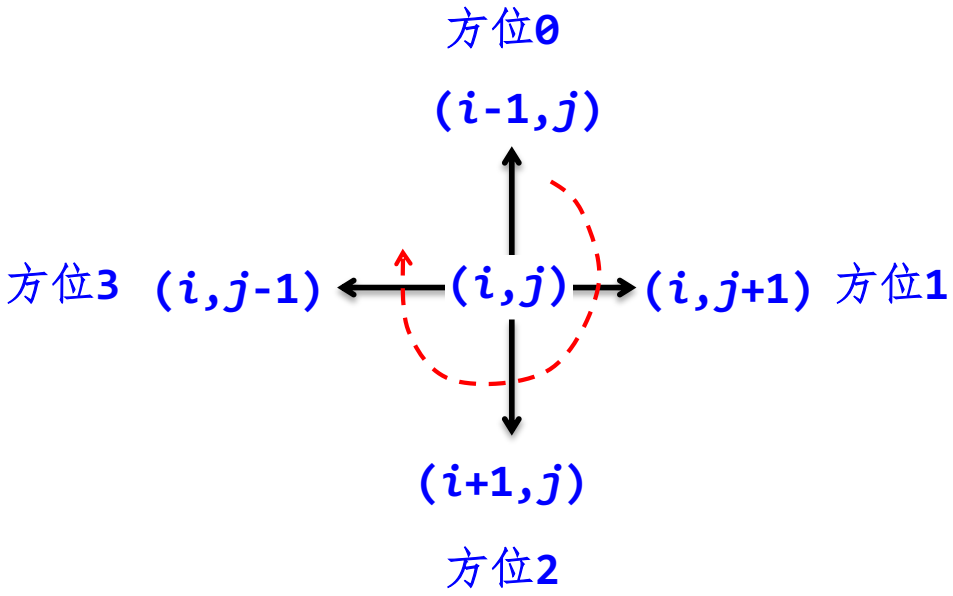
设置一个迷宫数组mg，其中每个元素表示一个方块的状态，为0时表示对应方块是通道，为1时表示对应方块不可走。




```
int mg[M+2][N+2]=
{
   {1, 1,1,1,1,1,1,1,1,1,  1},
   {1, 0,0,1,0,0,0,1,0,    1},
   {1, 0,0,1,0,0,0,1,0,    1},
   {1, 0,0,0,0,1,1,0,0,    1},
   {1, 0,1,1,1,0,0,0,0,    1},
   {1, 0,0,0,1,0,0,0,0,    1},
   {1, 0,1,0,0,0,1,0,0,    1},
   {1, 0,1,1,1,0,1,1,0,    1},
   {1, 1,0,0,0,0,0,0,0,    1},
   {1, 1,1,1,1,1,1,1,1,1,  1}
};
```

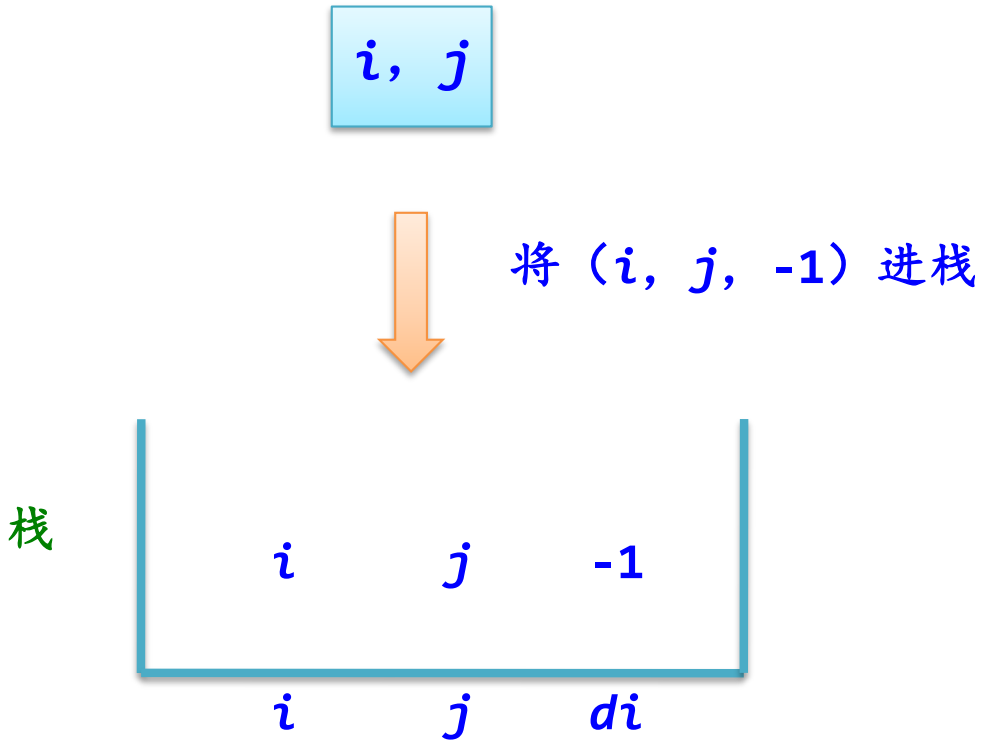
M×N

试探顺序



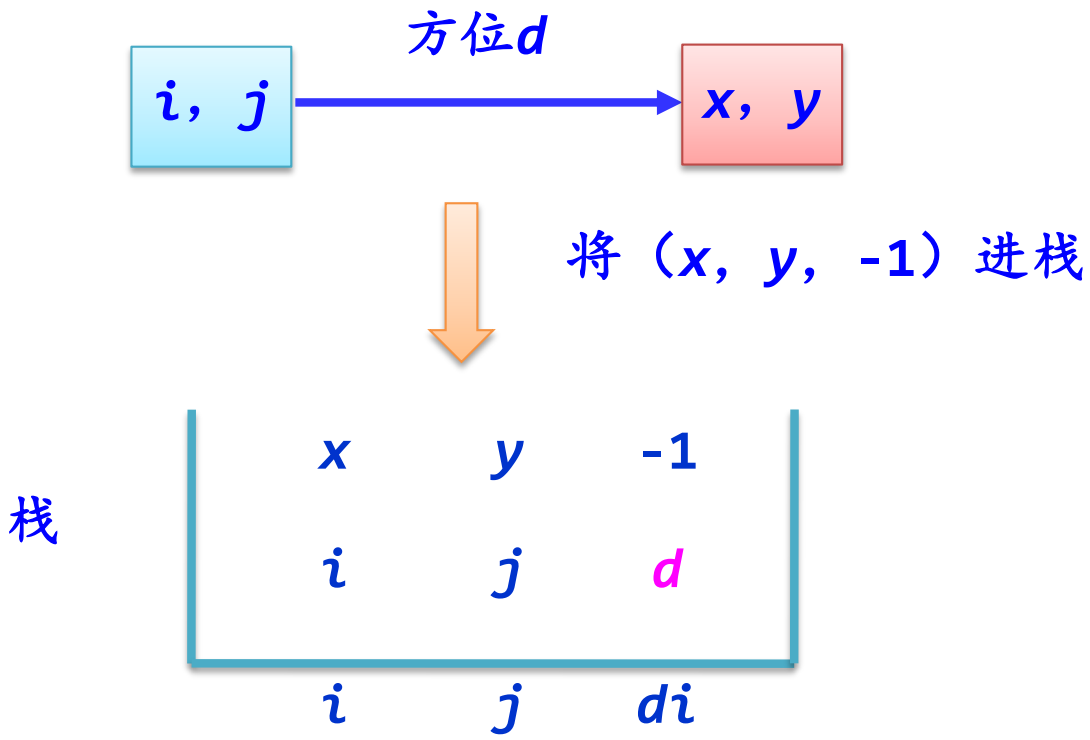

```
int dx[]={-1,0,1,0};          //x方向的偏移量
int dy[]={0,1,0,-1};          //y方向的偏移量
```

初始时，入口（i，j）作为当前方块。

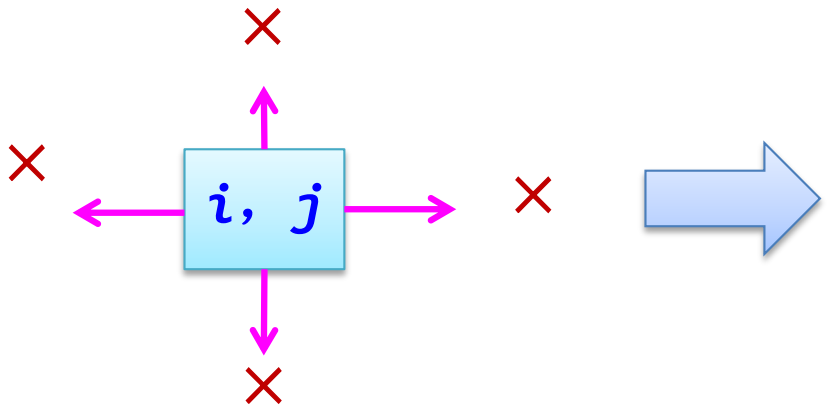


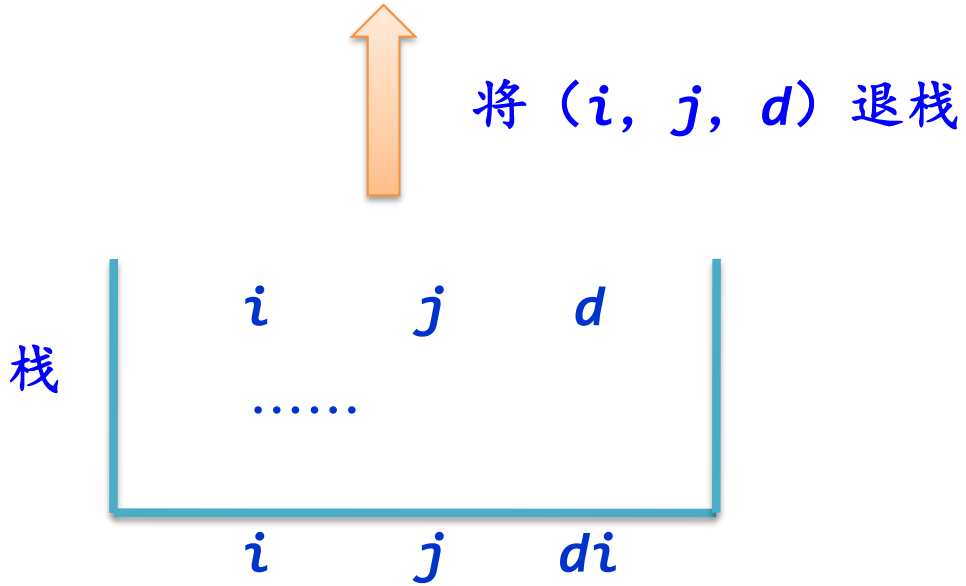
所有走过的方块都会进栈！

如果一个当前方块（i，j）找到一个相邻可走方块（x，y），就继续从（x，y）走下去。

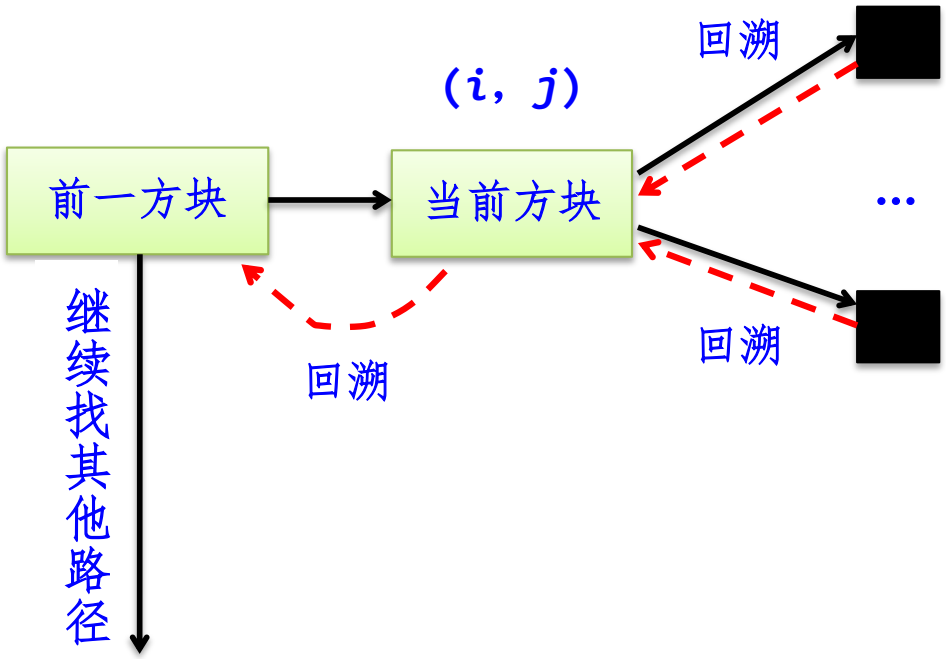


如果一个当前方块（i，j）没有找到任何相邻可走方块，表示此时无路可走，将其退栈。





迷宫问题的搜索过程

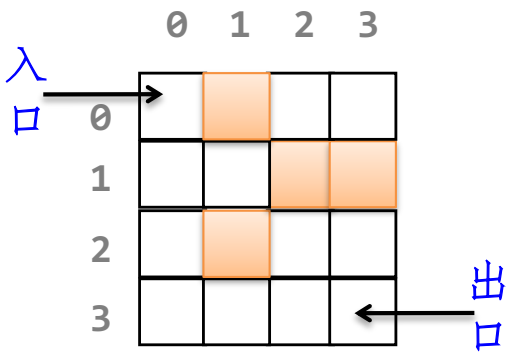


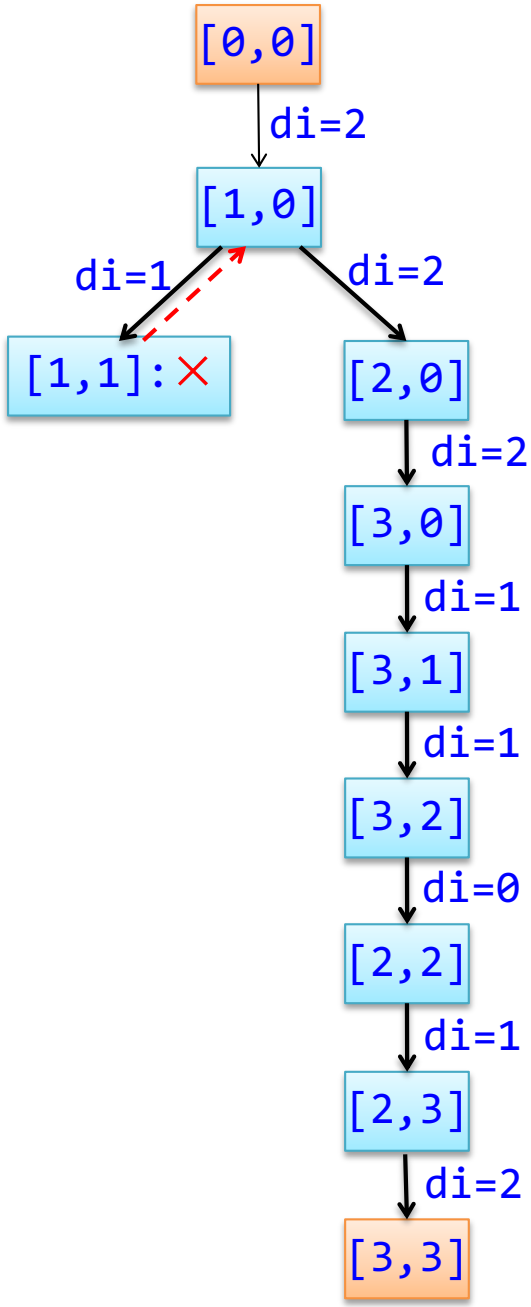
- 用栈记录走过的路径
- 路径：由方块和方块之间的走向（方位）构成

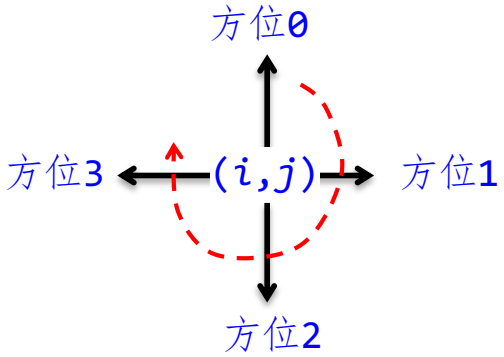


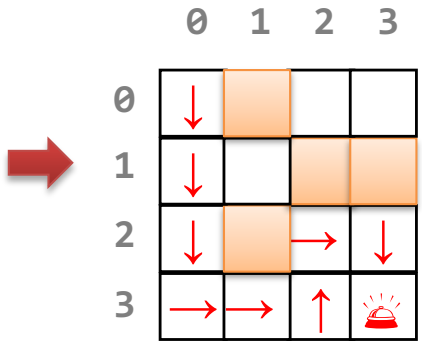



```
struct Box                                  //方块类型
{   int i;                                  //方块的行号
    int j;                                  //方块的列号
    int di;                                 //di是下一个相邻可走方位的方位号
    Box() {}                                //构造函数
    Box(int i1,int j1,int d1):
        i(i1),j(j1),di(d1) {}               //重载构造函数
};
```

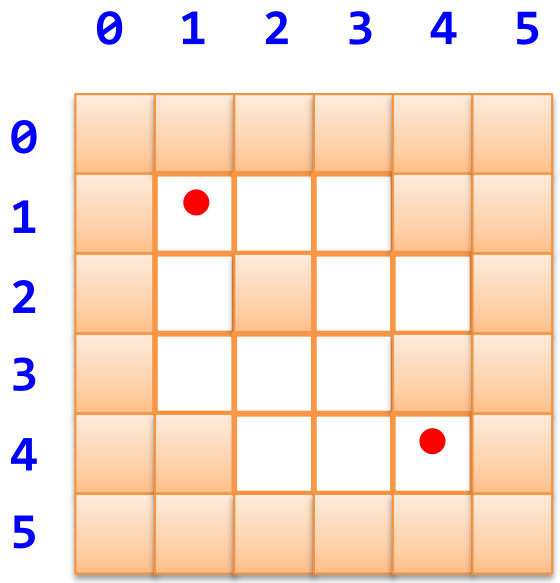

0 1 2 3
入口
0
1
2
3
出口

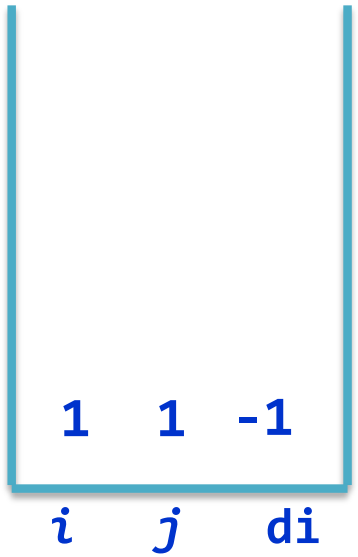

[0,0]
di=2
[1,0]
di=1
di=2
[1,1]:×
[2,0]
di=2
[3,0]
di=1
[3,1]
di=1
[3,2]
di=0
[2,2]
di=1
[2,3]
di=2
[3,3]


方位0
方位3
(i,j)
方位1
方位2


0 1 2 3
0
1
2
3

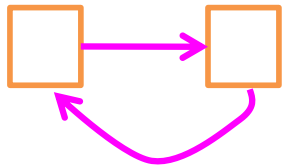
```
bool mgpath(int xi,int yi,int xe,int ye)
//求一条从(xi,yi)到(xe,ye)的迷宫路径
{  int i,j,di,i1,j1;
   bool find;
   Box b,b1;
   stack<Box> st;                    //建立一个栈
   b=Box(xi,yi,-1);
   st.push(b);                       //入口方块进栈
   mg[xi][yi]=-1;                    //为避免来回找相邻方块置mg值为-1
```





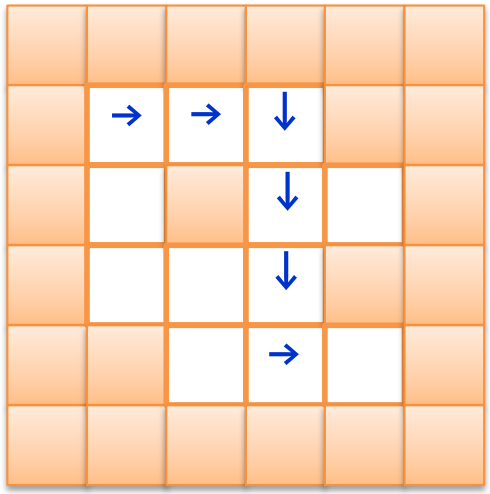
一个栈

为了避免重复，当一个方块进栈时，将迷宫值改为-1



```
while (!st.empty())                          //栈不空时循环
   {  b=st.top();                            //取栈顶方块,称为当前方块
      if (b.i==xe && b.j==ye)                //输出栈中所有方块构成一条路径
      {  disppath(st);
         return true;                        //找到一条路径后返回true
      }
      find=false;                            //否则继续找路径
      di=b.di;
      while (di<3 && find==false)            //找b的一个相邻可走方块
      {  di++;                               //找下一个方位的相邻方块
         i=b.i+dx[di];                       //找b的di方位的相邻方块(i,j)
         j=b.j+dy[di];
         if (i>=0 && i<m && j>=0 && j<n && mg[i][j]==0)
           find=true;                        //(i,j)方块有效且可走
      }
      if (find)                              //找到一个相邻可走方块(i,j)
      {  st.top().di=di;                     修改栈顶方块的di为新值
         b1=Box(i,j,-1);                     //建立相邻可走方块(i,j)的对象b1
         st.push(b1);                        //b1进栈
         mg[i][j]=-1;                        //为避免来回找相邻方块置mg值为-1
      }
      else                                   //栈顶方块没有找到任何相邻可走方块
      {  mg[b.i][b.j]=0;                     //恢复栈顶方块的迷宫值
         st.pop();                           //将栈顶方块退栈
      }
   }
   return false;                  //没有找到迷宫路径,返回false
}
```

```
while (!st.empty())                    //栈不空时循环
   {  b=st.top();                              //取栈顶方块,称为当前方块
      if (b.i==xe && b.j==ye)                  //输出栈中所有方块构成一条路径
      {  disppath(st);
         return true;                          //找到一条路径后返回true
      }
```

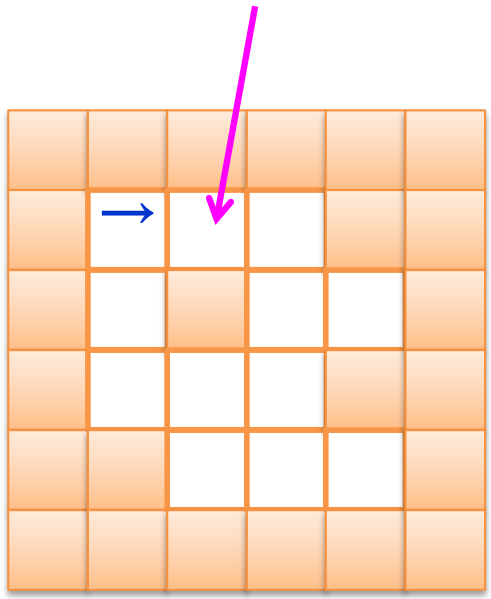




i	j	di
4	4	-1
4	3	1
3	3	1
2	3	2
1	3	2
1	2	1
1	1	1

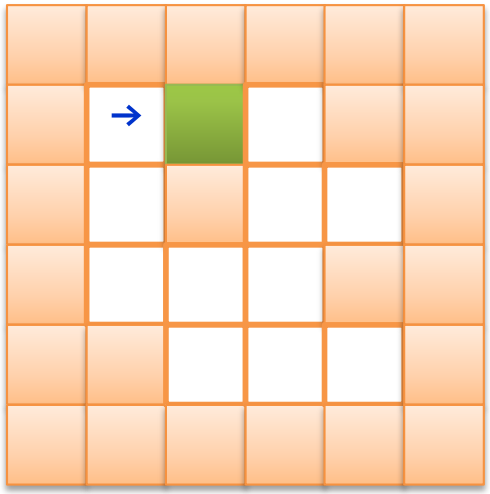
```
find=false;                                  //否则继续找路径
di=b.di;
while (di<3 && find==false)                  //找b的一个相邻可走方块
{  di++;                                     //找下一个方位的相邻方块
   i=b.i+dx[di];                             //找b的di方位的相邻方块(i,j)
   j=b.j+dy[di];
   if (i>=0 && i<m && j>=0 && j<n && mg[i][j]==0)
      find=true;                             //(i,j)方块有效且可走
}
```

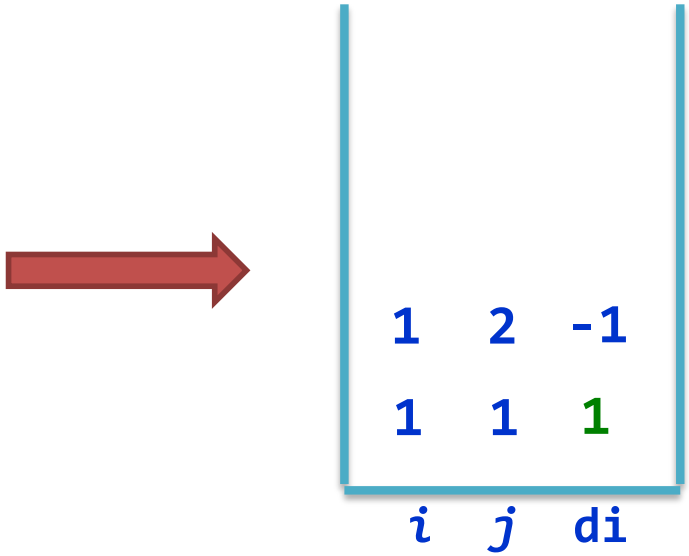
从入口（1,1）出发找到一个可走方块（1, 2）



```
if (find)                          //找到一个相邻可走方块(i,j)
{  st.top().di=di;                 //修改栈顶方块的di为新值
   b1=Box(i,j,-1);                 //建立相邻可走方块(i,j)的对象b1
   st.push(b1);                    //b1进栈
   mg[i][j]=-1;                    //为避免来回找相邻方块置mg值为-1
}
```

从入口（1,1）出发找到一个可走方块（1，2）:将（1，2，-1）进栈

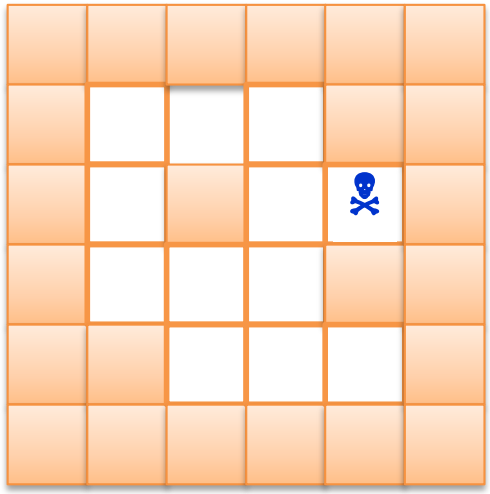




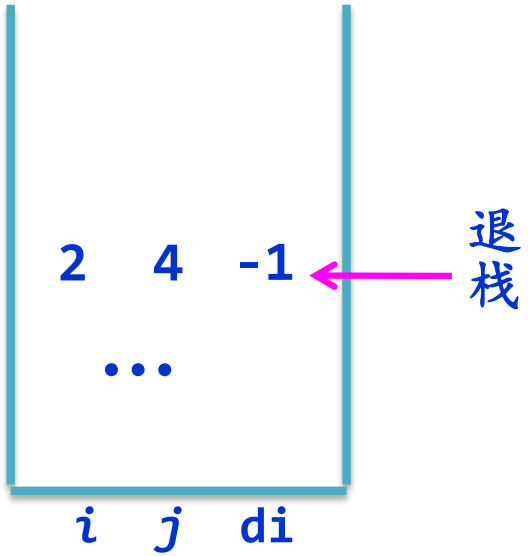
一个栈

```
        else                                    //栈顶方块没有找到任何相邻可走方块
        {  mg[b.i][b.j]=0;                      //恢复栈顶方块的迷宫值
           st.pop();                            //将栈顶方块退栈
        }
    }
    return false;                               //没有找到迷宫路径,返回false
}
```

（2,4）方块没有通路







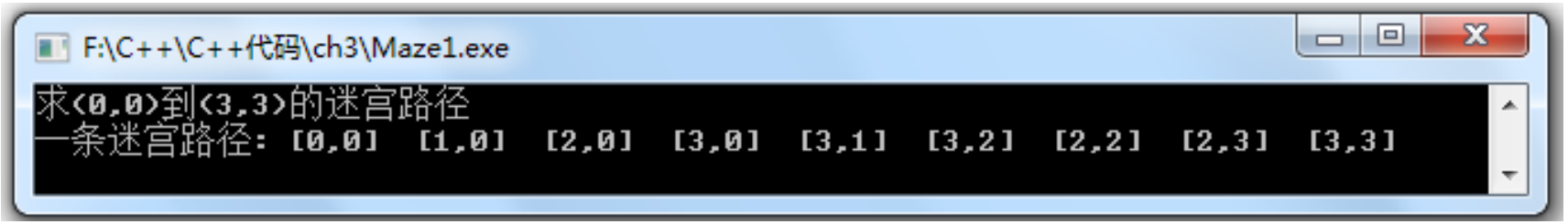
一个栈

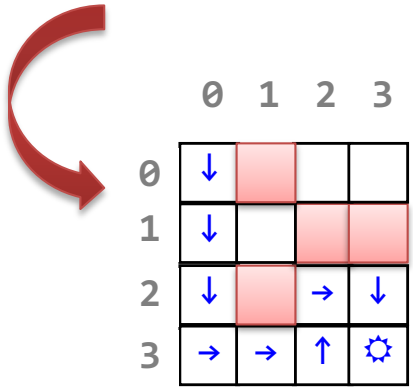
```
void disppath(stack<Box>& st)  //输出栈中所有方块构成一条迷宫路径
{  Box b;
   vector<Box> apath;                    //存放一条迷宫路径
   while (!st.empty())                   //出栈所有的方块
   {  b=st.top(); st.pop();
      apath.push_back(b);
   }
   reverse(apath.begin(),apath.end());//逆置apath(也可反向输出apath)
   cout << "一条迷宫路径: ";
   for (int i=0;i<apath.size();i++)
      cout << "[" << apath[i].i << "," << apath[i].j << "]  ";
   cout << endl;
}
```

设计主程序

```
int main()
{   int xi=0,yi=0,xe=3,ye=3;
    printf("求(%d,%d)到(%d,%d)的迷宫路径\n",xi,yi,xe,ye);
    if (!mgpath(xi,yi,xe,ye))
        cout << "不存在迷宫路径\n";
    return 0;
}
```

程序验证





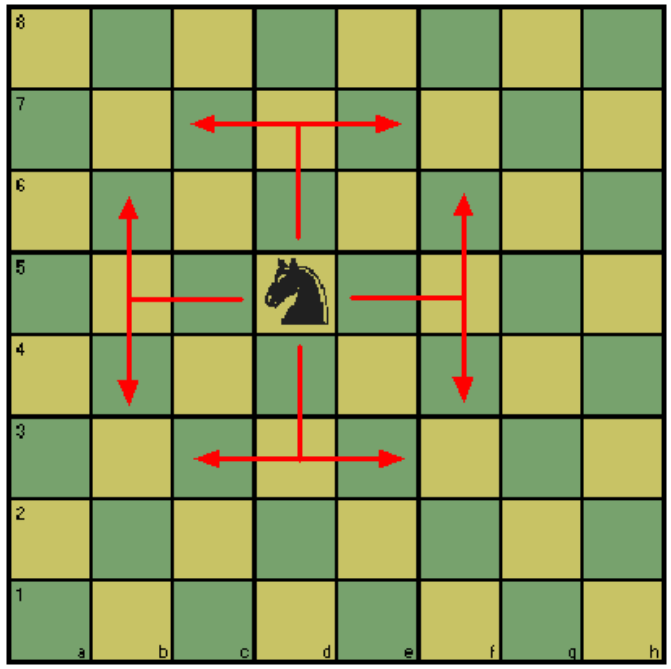


- 为什么找到的路径不一定是最短路径？
- 如何求所有的迷宫路径？

骑士最短路径

国际象棋中的骑士，类似于中国象棋中的马，有八种走法(如下图所示)：

1. 左走一格向前走两格；
2. 右走一格向前走两格;
3. 左走一格向后走两格；
4. 右走一格向后走两格
5. 左走两格向前走一格；
6. 左走两格向后走一格
7. 右走两格向前走一格
8. 右走两格向后走一格



给定8*8的棋盘，用字母a~h表示列，数字1~8表示行。棋盘上任意一个格子的位置可由一个字母和一个数字组成的字符串表示。例如，图示中骑士所在位置为d4，它一步可达的8个位置分别为c2, e2,c6, e6, b3, b5, f3, f5。你的任务是写一个程序，对于棋盘上任一两个位置x和y，计算骑士从位置x动到位置y最少需要移动多少步。

1．名人问题

给定一个大小为n×n的方阵 mat[][]，其中 mat[i][j]==1表示第i个人认识第j个人，mat[i][j]==0 表示第i个人不认识第j个人。请找出满足以下条件的“名人”：

名人的定义为：

- 被其他所有人认识；
- 不认识任何人（除了自己）。

若存在名人，返回其下标；否则返回 -1。

注意：题目保证对于所有的 i，都有 mat[i][i]==1。

输入： mat[][] = [[1, 1, 0],
[0, 1, 0],
[0, 1, 1]]

输出： 1

输入： mat[][] = [[1, 1],
[1, 1]]

输出：-1

栈解法 利用栈，通过两两比较排除非名人：

- 若一人认识另一人，则前者被排除；
- 若一人不认识另一人，则后者被排除。

重复此过程直至栈中仅剩一个候选，最后验证该候选是否符合名人条件。

分步实现：

1. 创建一个栈，并将所有人的编号压入栈中；
2. 循环执行直至栈中元素数量大于1：
   - 弹出栈顶的两个元素（记为A和B）；
   - 若A认识B，则A不可能是名人，将B重新压入栈；
   - 若A不认识B，则B不可能是名人，将A重新压入栈；
3. 将栈中剩余的唯一元素设为名人候选；
4. 遍历0到n-1的所有人，统计两个数值：
   - 认识该候选的人数；
   - 该候选认识的人数。
   - 若“认识候选的人数”等于n-1，且“候选认识的人数”等于0，则返回候选编号；否则返回-1。

双指针法：使用两个指针，一个从起始位置开始，一个从末尾位置开始。

假设起始位置的人是A，末尾位置的人是B：

- 若 A 认识 B，则 A 一定不是名人；
- 若 A 不认识 B，则 B 一定不是名人。

循环结束后，只会剩下一个下标作为名人候选。需再次遍历所有人，验证该候选是否真的符合名人条件。

分步实现：

1. 定义两个下标 i 和 j，初始值分别为 i = 0、j = n-1；
2. 循环执行直至 i < j 不成立：
   - 若 i 认识 j，则 i 不可能是名人，将 i 加 1（i++）；
   - 否则 j 不可能是名人，将 j 减 1（j--）；
3. 将 i 设为名人候选；
4. 最后再次遍历 0 到 n-1 的所有人，检查以下两种情况：
   - 候选是否认识某个人；
   - 是否存在某个人不认识候选。

   若满足任一情况则返回 -1；若循环结束后均不满足，则确认该候选为名人，返回其下标。

中缀转前缀

给定一个表示中缀表达式的字符串s，将其转换为前缀表示法。

输入： 3 + 4 * 2 / (1 - 5)

输出：+ 3 / * 4 2 - 1 5

解1：栈解法

- 扫描方向：从右到左遍历中缀表达式；
- 操作数处理：遇到数字直接加入结果列表；
- 运算符处理：用栈管理运算符，按优先级弹出栈顶运算符（优先级更高 / 相等且右结合时弹出），再压入当前运算符；
- 括号处理：遇到 ） 直接入栈，遇到 （ 弹出运算符直到 ）；
- 结果处理：遍历结束后弹出栈中剩余运算符，最后反转结果列表得到前缀表达式。

解2：借助后缀表达式 + 反转法

中缀转前缀可基于 “中缀转后缀”方法做少量修改实现，转换公式为：

前缀表达式 = reverse(postfix(reverse(中缀表达式)))

步骤如下：

1.反转中缀表达式（因前缀表达式从右到左求值，与后缀相反）；

2.反转时交换 (和)，以保证子表达式的分组逻辑正确；

3.使用标准的栈方法将修改后的表达式转换为后缀表达式；

4.反转得到的后缀表达式，最终结果即为前缀表达式。

步骤 1：反转中缀表达式 + 交换括号

- 前缀表达式的特性：求值/转换需从右到左扫描原中缀（因为运算符在操作数前）；
- 中缀转后缀的特性：求值/转换需从左到右扫描（因为运算符在操作数后）；
- 反转中缀后，原本“从右到左扫描原中缀”变成“从左到右扫描反转后的中缀”，和中缀转后缀的扫描方向完全一致，无需修改扫描逻辑。
- 括号交换：反转中缀会导致括号的分组逻辑颠倒，例如：原中缀 (3+4)*2 反转后是 2*)4+3(，此时括号变成了) 在前、(在后，若不交换，中缀转后缀的括号处理（识别 (入栈、) 出栈）会完全错误。

步骤 2：反转后的中缀转后缀

- 这一步的优先级规则和中缀转后缀类似（不完全一致）：仅当栈顶运算符优先级严格大于当前运算符时才弹出（等于时不弹出），这保证了反转后原左结合性的正确处理。
- 因为反转后的中缀扫描方向是 “右到左”，同级运算符（+-、*/）后遇到的（当前）是左边的操作符，需要先处理。

步骤 3：反转后缀结果得到前缀

反转后的中缀转出来的后缀，本质是 “目标前缀的逆序”，反转后即可得到正确的前缀表达式。

```
中缀：          3 + 4 * 2 / (1 - 5)
反转：          (5 - 1) / 2 * 4 + 3
后缀：          5   1 - 2   4 * / 3  +
反转（前缀）：+ 3 / * 4  2  -  1  5
```

后缀转中缀（带括号版）

给定一个表示后缀表达式的字符串s，将其转换为中缀表示法。

输入： 3 4 2 * 1 5 - / +

输出： ((3+((4 * 2)/(1 - 5))))

栈解法

1. 初始化栈：栈用于存储操作数和中间拼接的中缀表达式字符串；
2. 扫描后缀表达式
   - 遇到操作数（数字）：直接压入栈中；
   - 遇到运算符：从栈中弹出两个操作数（注意顺序：先弹出的是右操作数，后弹出的是左操作数）；
   - 拼接中缀字符串：格式为(左操作数 运算符 右操作数)，将拼接结果重新压入栈；
3. 遍历结束后，栈中仅剩一个字符串，即为中缀表达式

表达式“3 4 2 * 1 5 - / +”转换成中缀表达式（带括号）的过程

ch	操 作	栈
3	3进栈	3
4	4进栈	3 4
2	2进栈	3 4 2
*	弹出 2、4 → 拼接 (4 * 2) → 进栈	3 (4 * 2)
1	1进栈	3 (4 * 2) 1
5	5进栈	3 (4 * 2) 1 5
-	弹出 5、1 → 拼接 (1 - 5) → 进栈	3 (4 * 2) (1 - 5)
/	弹出 (1 - 5)、(4 * 2) → 拼接 ((4 * 2) / (1 - 5)) → 压栈	3 ((4 * 2)/(1 - 5))
+	弹出 ((4 * 2) / (1 - 5))、3 → 拼接 (3 + ((4 * 2) / (1 - 5))) → 压栈	(3 + ((4 * 2)/(1 - 5)))

后缀转中缀（去括号版）

给定一个表示后缀表达式的字符串s，将其转换为中缀表示法。

输入：　　3　4　2 *　1　5 - / +

输出(带括号)：　((3+((4 * 2)/ (1 - 5))))

输出(去括号)：　3 + 4 * 2 / (1 - 5)

后缀转中缀去冗余括号：

为每个中间表达式记录其“主导运算符”和优先级，拼接时仅在“子表达式优先级小于父运算符优先级”或“优先级相等但结合性不匹配”时添加括号。

1. 定义表达式结构体：

```
struct ExprInfo {
        string expr; // 表达式字符串
        char op; // 主导运算符（无则为'\0'）
        int priority; //
};
```

2. 优先级规则：

* / 优先级 = 2; +/- 优先级 = 1; 操作数优先级 = 0;

后缀转中缀（去括号版）

后缀转中缀去冗余括号：

3.括号添加规则（拼接 (左操作数 op 右操作数)时）：

- 左子表达式：仅当 左子表达式优先级 < op优先级 加括号；
- 右子表达式：
  - ✓ 若 右子表达式优先级 < op优先级 加括号；
  - ✓ 若 右子表达式优先级 == op优先级 且 op是左结合(如+-*/) 加括号（
    - ➢ 如 a + (b + c) 无需括号，但 a / (b / c) 需括号）；
  - ✓ 优先级相等且右结合（如 ^）→ 不加括号（本题暂不涉及）；

后缀转中缀（去括号版）

后缀转中缀去冗余括号：

优化后核心步骤

1. 栈存储 **ExprInfo** 而非纯字符串，记录每个表达式的主导运算符和优先级；
2. 扫描后缀表达式，遇到操作数时，构造“无运算符、优先级 0”的 **ExprInfo** 压栈；
3. 遇到运算符时，弹出右|左 **ExprInfo**，根据优先级规则判断是否为左右子表达式加括号；
4. 拼接新的 **ExprInfo**（记录当前运算符和优先级），压回栈；
5. 最终栈中表达式即为无冗余括号的中缀表达式。

表达式“3 4 2 * 1 5 - / +”转换成中缀表达式（去括号）的过程

ch	操 作	栈
3	(“3”, ‘\0’, 0)进栈	(“3”, ‘\0’, 0)
4	(“4”, ‘\0’, 0)进栈	(“3”, ‘\0’, 0) (“4”, ‘\0’, 0)
2	(“2”, ‘\0’, 0)进栈	(“3”, ‘\0’, 0) (“4”, ‘\0’, 0) (“2”, ‘\0’, 0)
*	弹出2、4，左右都是操作数(优先级0）无需括号，拼接后入栈	(“3”, ‘\0’, 0) (“4*2”, ‘*’, 2)
1	(“1”, ‘\0’, 0)进栈	(“3”, ‘\0’, 0) (“4*2”, ‘*’, 2) (“1”, ‘\0’, 0)
5	(“5”, ‘\0’, 0)进栈	(“3”, ‘\0’, 0) (“4*2”, ‘*’, 2) (“1”, ‘\0’, 0) (“5”, ‘\0’, 0)
-	弹出5、1，左右都是操作数(优先级0）无需括号，拼接后入栈	(“3”, ‘\0’, 0) (“4*2”, ‘*’, 2) (“1-5”, ‘-’, 1)
/	弹出1-5、4*2，左子表达式*优先级= op(/)无需括号，右子表达式-优先级<op需加括号	(“3”, ‘\0’, 0) (“4*2/(1-5)”, ‘/’, 2)
+	弹出4*2/(1-5)、3，左子表达式是操作数无需括号，右子表达式/优先级>op，无需加括号	(“3+4*2/(1-5)”, ‘+’, 1)